ICPC　CCPC

ACM 区域赛
算法与题型精讲

模型分析与 C++17 实现

从基础算法到区域赛题解



精编修订版

2026 年 10 月

前言

本书面向已有 C++ 基础、正在准备 ICPC 与 CCPC 区域赛的读者，围绕常用算法、核心模板和典型题展开。每个专题先说明模型与适用条件，再给出实现；例题进一步解释题意如何转化为状态、约束与可证明的算法。

全书共十二章，依次讨论枚举与贪心、二分与区间、动态规划、图与树、字符串、数论、线性代数、概率博弈和几何。常用方法配有可复制的 C++17 代码，容易混淆的状态与边界通过推演图说明。例题中的手工数据用于解释解法，除特别注明外，不等同于原题样例。

复习时可从需要补足的专题开始：先确认状态与适用条件，再核对代码的下标、数值范围和边界，最后独立完成例题。学习结果应体现为能够识别模型、说明正确性并稳定实现。

代码约定

核心代码面向 GCC 的 C++17 环境，共用第 1 章的 ll、i128 与 INF 定义。每节明确自己的下标与区间约定；跨节组合时先做坐标适配。不同模板的同名函数应放入独立命名空间。高精度使用 Boost；深递归版本已标明栈风险。代码为可编辑文本，可直接复制，不包含题目的完整输入输出。

题面与命题组题解链接随相应例题给出。基础模板题用于检验实现，区域赛例题用于训练模型转换。

目录

第 1 章 算法模型与实现约定.....1

1.1 比赛程序骨架与整数边界.....1
1.2 异或消去与位图.....2
1.3 从数据范围选择算法.....2
1.4 容器排序与实现规范.....3
1.5 整数除法与判定边界.....3

第 2 章 枚举分治与贪心.....4

2.1 归并排序与逆序对.....4
2.2 回溯位压缩与 N 皇后.....5
2.3 折半枚举.....5
2.4 区间调度与反悔堆.....6
2.5 构造题与交换论证.....7
2.6 典型题解析.....7

第 3 章 二分前缀与滑动窗口.....10

3.1 整数二分与二分答案.....10
3.2 前缀状态与子数组计数.....11
3.3 二维前缀和与矩形差分.....11
3.4 等差数列二阶差分.....12
3.5 双指针与至多 k 类计数.....13
3.6 可删除的中位数与绝对差代价.....14
3.7 典型题解析.....15

第 4 章 区间数据结构与离线方法.....17

4.1 单调栈与贡献边界.....17
4.2 单调队列与滑动极值.....17
4.3 树状数组与动态前缀.....18
4.4 树状数组的区间修改与第 k 小查询.....19
4.5 单点修改与区间最大值.....20
4.6 区间加与区间和的懒标记线段树.....21
4.7 静态 RMQ 的 Sparse Table.....22
4.8 离线查询与普通莫队.....23
4.9 结构选型与摊还分析.....24
4.10 典型题解析.....24

第 5 章 动态规划与状态压缩.....28

5.1 Kadane 与两个不重叠子段.....28
5.2 LIS 与双序列动态规划.....28
5.3 区间 DP 与最长回文子序列.....29

5.4 0 1 背包与完全背包.....30
5.5 多重背包与分组背包.....30
5.6 集合状压 DP 的基础骨架.....31
5.7 集合与末尾共同构成的状态.....32
5.8 子集枚举与 SOS DP.....32
5.9 数位 DP 的双边界状态.....33
5.10 转移设计与优化边界.....34
5.11 多维状态的典型模型.....34
5.12 典型题解析.....35

第 6 章 连通性最短路与生成树.....38

6.1 并查集与离线连通.....38
6.2 带奇偶关系的并查集.....39
6.3 BFS 多源搜索与洪水填充.....39
6.4 0 1 BFS.....40
6.5 Dijkstra 与状态扩展.....41
6.6 Bellman Ford 与 Floyd.....42
6.7 Prim 与 Kruskal 最小生成树.....42
6.8 离线阈值与状态图建模.....44
6.9 典型题解析.....44

第 7 章 拓扑缩点匹配与网络流.....46

7.1 拓扑排序与 DAG DP.....46
7.2 强连通分量与缩点 DAG.....46
7.3 桥与割点的 lowlink 判定.....48
7.4 Hopcroft Karp 二分图匹配.....49
7.5 最大流与最小割建模.....50
7.6 二元逻辑约束与 2 SAT.....52
7.7 图模型的适用边界.....52
7.8 典型题解析.....53

第 8 章 树上动态规划与路径维护.....56

8.1 迭代树序与不相邻点最大和.....56
8.2 倍增 LCA 与树上距离.....56
8.3 DFS 序与重链剖分.....58
8.4 换根 DP 与树重心.....59
8.5 树结构与边界处理.....60
8.6 典型题解析.....61

第 9 章 字符串匹配与回文结构.....63

9.1 KMP 前缀函数与匹配.....63
9.2 Z 函数与前缀匹配.....64
9.3 双模字符串哈希.....64
9.4 字典树的词频与前缀频次.....65
9.5 AC 自动机与多模式统计.....66
9.6 二进制 Trie 与最大异或.....67

9.7 Manacher 与全部回文半径.....68
9.8 解析与自动机组合.....69
9.9 典型题解析.....69

第 10 章 数论组合与高精度计算.....72

10.1 模乘快速幂与矩阵快速幂.....72
10.2 素数筛素性检验与因数分解.....73
10.3 欧拉函数与莫比乌斯函数.....74
10.4 扩展欧几里得与扩展 CRT.....75
10.5 组合数阶乘逆元与 Catalan.....76
10.6 Lucas 定理与整除分块.....77
10.7 容斥与二项式反演.....78
10.8 高精度整数与精确算术.....79
10.9 常见计数公式与置换结构.....79
10.10 典型题解析.....80

第 11 章 线性代数概率与博弈.....82

11.1 异或线性基.....82
11.2 实数高斯消元.....83
11.3 GF2 高斯消元的位压缩版本.....84
11.4 素数模高斯消元.....85
11.5 0 1 分数规划.....86
11.6 概率 DP 与期望方程.....86
11.7 SG 与经典取石子游戏.....87
11.8 有环图的胜负逆推.....88
11.9 从规则选择博弈与线性模型.....89
11.10 典型题解析.....89

第 12 章 基础几何与凸包.....91

12.1 整数向量与凸包.....91
12.2 线段相交与旋转卡壳.....92
12.3 极角面积与退化情况.....93
12.4 典型题解析.....93

第 1 章 算法模型与实现约定

算法模板的使用以模型成立为前提。本章统一程序骨架、整数类型与区间约定，并说明如何根据状态数、转移数和数值范围判断实现是否可行。

1.1 比赛程序骨架与整数边界

下标和节点编号一般用 int，累计和、乘积和距离用 long long。乘法先提升到 __int128 再计算。需要修改实参用引用；只读大对象用 const 引用，避免复制。

标准输出流没有 __int128 的直接输出接口，可用 cout << text128(x)。转换负数时先进入无符号域，再求其大小，以覆盖最小负数这一特殊边界。

复杂度与接口 读入与输出 O(输入长度)；骨架不依赖全局数组。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
const ll INF = (1LL << 62);
string text128(i128 x) {
    using u128 = __uint128_t;
    bool neg = x < 0;
    u128 v = neg ? u128(0) - u128(x) : u128(x);
    string s;
    do {
        s.push_back(char('0' + v % 10));
        v /= 10;
    } while (v);
    if (neg)
        s.push_back('-');
    reverse(s.begin(), s.end());
    return s;
}
void solve() {
    // 每组数据的容器在这里创建或清空。
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    cin >> T; // 单组数据时删除这一行。
    while (T--)
        solve();
    return 0;
}
```

实现要点 不要混用关闭同步后的 cin 与 scanf。交互询问要 flush；非交互输出优先使用换行字符。移位次数必须小于类型位宽。

位运算速记：x & (x−1) 删除最低的 1；x & −x 提取最低的 1；popcount 统计 1 的数量。用 unsigned 类型进行位图和二进制模型操作更稳妥。x>0 且 (x & (x−1))=0 才是二的幂。

1.2 异或消去与位图

相同整数满足 x xor x=0。若恰有两个数出现奇数次，总异或值 z 就是这两个数的异或；选取 z 的任一置位分组，两数必落在不同组中，再分别异或即可。位图则以一位记录一个有限状态，适用于可控值域。

复杂度与接口 异或消去 O(n)，额外空间 O(1)；位图占用约 N/8 字节。

```
pair<unsigned long long, unsigned long long>
two_odd(const vector<unsigned long long> &a) {
    unsigned long long s = 0, x = 0;
    for (auto v : a)
        s ^= v;
    // 前提：恰有两个不同的奇数次元素，所以 s != 0。
    auto bit = s & (0ULL - s);
    for (auto v : a)
        if (v & bit)
            x ^= v;
    return {x, s ^ x};
}
struct Bitmap {
    vector<unsigned long long> b;
    explicit Bitmap(int n) : b((n + 63) / 64) {}
    void set(int x) { b[x >> 6] |= 1ULL << (x & 63); }
    void erase(int x) { b[x >> 6] &= ~(1ULL << (x & 63)); }
    bool has(int x) const { return b[x >> 6] >> (x & 63) & 1; }
};
```

实现要点 交换同一个对象时不能用异或交换。有符号溢出和负数左移不能当成可靠的补码技巧。

移位量必须小于类型位宽。位运算统一使用无符号类型，并按值域选择 32 位、64 位或 bitset；有符号溢出与负数左移不可作为实现依据。

1.3 从数据范围选择算法

总成本由状态数、每个状态的转移数及单次运算代价共同决定。n≤20 只是集合状态的规模提示；若再加入末尾位置或枚举下一项，时间与内存都需相应增加。扩展图、数位余数与自动机状态也应计入总规模。

主要规模	首先考虑	需要额外检查
n 约 20	子集枚举与状压	2^n 的状态大小与转移数
n 约 40	折半枚举	合并方式与内存
n 约几千	平方或部分立方 DP	时限与常数
n 达十万以上	排序 扫描 对数结构	总数据量与哈希风险

表中的范围仅供初步选型。最终仍需结合时限、语言与常数估算；多组输入按 Σn、Σm 计算总规模，高精度运算还需考虑整数位数。

1.4 容器排序与实现规范

vector 用于顺序存储，queue 用于 FIFO，priority_queue 用于最值候选，set 或 multiset 用于前后继与任意删除。unordered_map 的期望常数复杂度依赖散列；对抗输入可采用随机盐散列或改用排序与离散化。比较器必须满足严格弱序，不能写成 a≤b，也不要通过可能溢出的减法比较大小。

固定下标约定：原数组与 STL 通常从 0 开始，Fenwick 从 1 开始；闭区间 [l,r] 与半开区间 [l,r) 不混用。多组数据的容器放在 solve 内，全局计数、队列、Trie 节点与访问标记应清空实际使用范围。

乘法与坐标差都应在运算前提升类型。负余数先规范化，整数除法按需要采用 floor 或 ceil。INF 既要大于合法答案，也要为后续加法保留余量。

1.5 整数除法与判定边界

C++ 的整数除法向零截断，例如 −5/2 得到 −2，而数学向下取整是 −3。二分判定、等差操作起点与整点不等式经常需要精确的 floor 或 ceil。先计算商与余数，再根据余数和除数的符号调整，避免使用加上 b−1 的只适用于正数的写法。

```
i128 floor_div(ll a, ll b) {
    assert(b != 0);
    i128 q = i128(a) / b, r = i128(a) % b;
    if (r != 0 && ((r < 0) != (b < 0)))
        --q;
    return q;
}
i128 ceil_div(ll a, ll b) {
    assert(b != 0);
    i128 q = i128(a) / b, r = i128(a) % b;
    if (r != 0 && ((r < 0) == (b < 0)))
        ++q;
    return q;
}
```

先将 ll 输入提升为 i128 再相除，以覆盖最小值除以 −1 的结果。返回值缩回 ll 前应检查范围；阈值比较优先整理为整数不等式，避免舍入改变边界。

验证判定时，重点检查分界附近的 x−1、x、x+1，以及重复值、负数和极值。判定逻辑需先于二分验证。

第 2 章 枚举分治与贪心

枚举明确候选范围，分治拆分子问题，贪心利用可证明的选择次序。三类方法的共同出发点是完整描述解空间，再依据结构减少搜索；允许撤销旧选择时，可用堆维护当前最优集合。

2.1 归并排序与逆序对

归并时，若当前右值小于当前左值，它也小于左半区尚未输出的全部元素，因此可一次计入这些跨区逆序对。左右半区内部的逆序对递归统计，与跨区部分互不重叠。

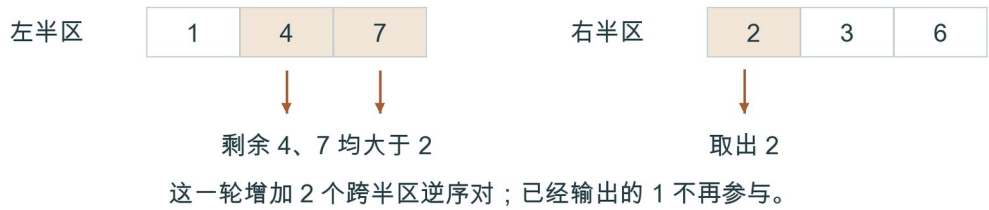


图 2.1 归并时批量统计跨区逆序对

复杂度与接口 时间 O(n log n)，额外空间 O(n)。

```
ll inversions(vector<ll> &a) {
    vector<ll> tmp(a.size());
    function<ll(int, int)> dfs = [&](int l, int r) -> ll {
        if (r - l <= 1)
            return 0;
        int m = (l + r) / 2;
        ll ans = dfs(l, m) + dfs(m, r);
        int i = l, j = m, k = l;
        while (i < m && j < r) {
            if (a[i] <= a[j])
                tmp[k++] = a[i++];
            else
                ans += m - i, tmp[k++] = a[j++];
        }
        while (i < m)
            tmp[k++] = a[i++];
        while (j < r)
            tmp[k++] = a[j++];
        copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        return ans;
    };
    return dfs(0, a.size());
}
```

实现要点 本实现统计 a[i]>a[j] 的普通逆序对。对于 a[i]>2a[j]，应在两个有序半区上另做双指针计数，再执行归并；倍乘前须提升类型。

归并还可累计跨区间的小和：当某个左值可贡献给一段右值时，一次乘入对应数量。排序与选择工具的比较见 2.5 节。

2.2 回溯位压缩与 N 皇后

回溯的核心是增量维护约束，并在离开分支时恢复状态。N 皇后按行选列，只需记列、左斜线、右斜线的占用；位运算能一次枚举当前所有可选列，减少检查常数。

复杂度与接口 搜索为指数级；位压缩减少每个状态的检查开销。

```
unsigned long long queens(int n) {
    assert(0 <= n && n < 32);
    unsigned int full = n ? (1U << n) - 1 : 0;
    function<unsigned long long(unsigned int, unsigned int, unsigned int)>
        dfs;
    dfs = [&](auto col, auto L, auto R) -> unsigned long long {
        if (col == full)
            return 1;
        auto ok = full & ~(col | L | R);
        unsigned long long ans = 0;
        while (ok) {
            auto p = ok & (0U - ok);
            ok -= p;
            ans += dfs(col | p, ((L | p) << 1) & full, (R | p) >> 1);
        }
        return ans;
    };
    return dfs(0, 0, 0);
}
```

实现要点 本实现要求 n<32，且答案可由 unsigned long long 保存。全排列与子集枚举在计入输出后分别为 O(n!·n)、O(2^n·n)，应先核对规模。

剪枝必须给出必要条件：不可能完成的分支才可删除。任意选择顺序、对称性减半和只找一个解各有不同计数语义，不可直接混用。

有重复元素的排列每层用集合去重，或排序后使用“相同值的前一个尚未使用时跳过”。组合与子序列枚举、汉诺塔、递归逆序栈和栈排序都是典型应用。Master 公式只适用于相同规模递归；T(n)=aT(n/b)+O(n^c) 若 log_b a>c，则为 O(n^(log_b a))；相等则为 O(n^c log n)；较小则为 O(n^c)。额外 log 因子需单独判断。

2.3 折半枚举

当 n 约为 40 时，直接枚举 2^n 个子集通常不可行，而分别处理两半的 2^(n/2) 个摘要仍可能接受。枚举后用排序、二分或双指针合并，以空间换取指数规模的下降。

复杂度与接口 时间 O($2^{n/2}$·n)，空间 O($2^{n/2}$)。

```
vector<ll> subset_sums(const vector<ll> &a) {
    vector<ll> s{0};
    for (ll x : a) {
        int n = s.size();
        for (int i = 0; i < n; i++)
            s.push_back(s[i] + x);
    }
    return s;
}
ll count_leq(const vector<ll> &a, ll W) {
```

```
    int m = a.size() / 2;
    auto L = subset_sums(vector<ll>(a.begin(), a.begin() + m));
    auto R = subset_sums(vector<ll>(a.begin() + m, a.end()));
    sort(R.begin(), R.end());
    ll ans = 0;
    for (ll x : L)
        ans += upper_bound(R.begin(), R.end(), W - x) - R.begin();
    return ans;
}
```

实现要点 有负数时不能用“当前和超目标就剪枝”。大量重复和必须保留频次；计数包含空子集，题目要求非空时要单独扣除。

合并方式取决于目标：存在性找补数，最大不超阈值找上界，计数要累计相同摘要的频次。可去重与不能去重的任务必须区分。

2.4 区间调度与反悔堆

反悔贪心允许在处理更晚的约束时，撤销此前最不合适的选择。按截止时间排序后维护已选任务总耗时，一旦超出当前截止时间就删去耗时最长任务，使留下的集合尽可能短。

复杂度与接口 时间 O(n log n)，空间 O(n)。

```
int max_courses(vector<pair<ll, ll>> jobs) {
    // pair 为 duration, deadline。
    sort(jobs.begin(), jobs.end(),
         [](auto a, auto b) { return a.second < b.second; });
    priority_queue<ll> q;
    ll t = 0;
    for (auto [d, ddl] : jobs) {
        t += d;
        q.push(d);
        if (t > ddl)
            t -= q.top(), q.pop();
    }
    return q.size();
}
ll huffman_cost(vector<ll> a) {
    priority_queue<ll, vector<ll>, greater<ll>> q(a.begin(), a.end());
    ll ans = 0;
    while (q.size() > 1) {
        ll x = q.top();
        q.pop();
        ll y = q.top();
        q.pop();
        ans += x + y;
        q.push(x + y);
    }
    return ans;
}
```

实现要点 区间端点能否相接决定 $\geqslant$ 或 $>$。课程模型要求时长为正且截止期是完成期限；不能直接套到有依赖或额外收益的任务。

交换论证要说明删除最长任务不会减少未来可安排数量。利润、耗时和截止时间角色不同；只有相同目标的可交换结构才支持同一堆策略。

2.5 构造题与交换论证

构造题先从总量、奇偶、颜色容量、连通性或度数建立必要条件，再寻找保持这些条件的局部操作。完整解法还需证明操作合法、过程终止，并覆盖全部可行输入。

贪心证明常用三种形式。交换论证把任意最优解改成符合当前选择的最优解；领先论证比较处理任意前缀后的进度；不变量论证说明每次操作维持可延伸的最优状态。区间调度选最早结束、Huffman 合并最小两项和反悔删最长任务，虽然都用排序或堆，证明对象各不相同。

排序工具应按任务选择：随机三路快排适合大量重复值，nth_element 适合平均线性的第 k 小选择，归并适合稳定排序与跨区贡献统计，基数排序适合固定长度整数键。

2.6 典型题解析

例题 2.1 Strips 无重叠覆盖与倒序修正

2024 ICPC 南京 K

问题 数轴格子中有红格和黑格，用最少互不重叠、长度为 k 的纸条盖住所有红格，不能盖黑格。

解法 黑格把问题分成独立空白段 [L,R]。每段排序红点，遇到尚未覆盖的最左红点，就从它向右铺一条。

忽略右边界时这个贪心最小化纸条数。只有末纸条可能越界，把它移到 R−k+1。

如果末纸条与前条重叠，依次向左修正：前条左端点取 min(原左端点,后条左端点−k)。

若第一条越过 L 则无解，否则所有红格仍被覆盖且无重叠。每段无红点直接略过。

正确性 最左未覆盖红点必须被某条纸条覆盖；让该纸条尽量向右不损失对后续点的覆盖。倒序移动只是压缩多余间隙，已覆盖红点不会落到相邻纸条间的空缺中。若最少纸条总长度都放不下该段，任何更多纸条更不可能。

示例推演 段 [1,10]、k=3，红点 2、5、9。初始纸条 [2,4]、[5,7]、[9,11]，末条改为 [8,10] 即合法。若段只有 [1,7] 而需要 3 条，总长度 9>7，则无解。

实现要点 纸条不能重叠，不能像普通覆盖题一样只选最右端点。黑格处的段边界是黑格前后一个位置，格点长度为 R−L+1。

复杂度 O((n+m) log(n+m))，排序后线性构造。

练习建议 枚举短段全部纸条放置，验证最少数量与倒序调整；特别测 k 大于段长及段端点恰好是红点。

原题与解法依据 2024 ICPC 南京 K

例题 2.2 PTSD 配对贪心的交换论证

2021 CCPC 桂林 I

问题 士兵战力为 1..n，可分成任意组。带标记士兵若在其组恰是第二强，就贡献自己的战力，最大化贡献和。

解法 每个有贡献的人必须有一个更强的人与之同组。每组最多一个第二强，因此可把有效部分抽成不相交二人配对。

从战力大到小扫描，cnt 表示已经遇到且尚未配对的更强士兵。

当前有标记且 cnt>0，就让他贡献、cnt--；否则把他作为以后较弱标记士兵的强队友，cnt++。

多余士兵可以独立成组，不需要为所有士兵强制配对。

正确性 当前标记士兵是尚未决定的人中收益最大的可选贡献者。若最优解暂不让他贡献，却让一个更弱者与当前可用强者配对，替换后可行且收益不减。后续只关心强者数量，无需保留身份。

示例推演 n=5、标记为 [0,1,0,1,0]。扫描 5 留作强者；4 贡献 4；3 留作强者；2 贡献 2，总计 6。

```
ll ptsd(const string &s) {
    ll ans = 0;
    int available = 0;
    for (int i = (int)s.size() - 1; i >= 0; i--) {
        if (s[i] == '1' && available)
            ans += i + 1, --available;
        else
            ++available;
    }
    return ans;
}
```

实现要点 不要先把所有带标记者组成一组；只有第二强贡献。普通排序后相邻配对也不总最优，状态 cnt 必须反映哪些强者仍可用。

复杂度 O(n) 时间，O(1) 额外空间。

练习建议 n≤8 枚举集合分割核对贪心；将战力改为任意互异正数，再检验论证是否仍适用。

原题与解法依据 2021 CCPC 桂林 I

例题 2.3 New Energy Vehicle 根据下一次补给消耗资源

2024 CCPC 哈尔滨 J

问题 车有多种电池，每种容量上限不同，耗任意电池一单位可向前一单位。充电站只给指定电池充电，起初满电，求最远行驶距离。

解法 预处理每种电池下一次能够被补充的站。行驶到下一个站时，优先消耗下一次补给更早的电池。

若先消耗更晚才能补给的电池，而早补给的电池还满着，到早站时会浪费充电容量；交换这两部分消耗不会变差。

用堆按下一补给位置维护电量块。当前区间距离会整块消耗若干电池，最后只部分消耗堆顶。

到站更新该电池的剩余量与下一补给信息；若用惰性堆条目，附版本号忽略过时记录。

正确性 两种资源消耗互换只改变未来可充入的空间，早补给资源先消耗能够释放更早的空间，而晚补给资源保留下来不会减少当前可达距离。连续执行交换得到贪心顺序。

示例推演 A、B 各容量 5，下一站在距离 4，只能补 A。应先耗 A 的 4，再补回；若先耗 B，A 仍满、充电被浪费，站后总可用电量少 4。

实现要点 选择当前剩余最多的电池没有交换论证。重复遇到同类充电站时下一补给位置会变化；堆内旧优先级必须更新或惰性失效。

复杂度 排序站点及堆维护 O((n+m) log(n+m))，空间 O(n+m)。

练习建议 先实现 O(nm) 扫描所有电池的贪心，再用堆优化，逐次检查两版电量与最远距离完全一致。

原题与解法依据 2024 CCPC 哈尔滨 J

例题 2.4 Finding Keywords 子序列阶段计数

2025 CCPC 济南 C

问题 目标 b 是长度 m 的排列，每个输入序列尽量选多条位置互不相交、恰等于 b 的子序列。

解法 记录目标每个值的排列位置 pos。扫描输入值 x，若不在目标中就忽略。

x 是目标第 p 项时，只能接在一个已经匹配 p−1 项的序列后面。若这样的序列存在，减少阶段 p−1 的数量，增加阶段 p。

p=1 时可以开始一条新序列；阶段 m 的数量就是答案。目标元素互异，因此每个 x 对应的阶段唯一。

不要把整条目标当作子串做 KMP，因为题目允许子序列中的位置有间隔。

正确性 目标值互异，一个值只服务于一个阶段。若可推进却暂缓使用当前值，可把以后用于同阶段的同值换到当前位置，不影响其他序列。因此尽早推进不减少最终完成条数。

示例推演 目标 [2,1,3]，输入 [2,2,1,3,1,3]。两个 2 各启动一条序列，之后依次补入 1 和 3，最终完成两条。

实现要点 目标为排列，保证单值对应唯一阶段；含重复值时这一映射失效，不能沿用相同转移顺序。

复杂度 O(输入总长度+m)，空间 O(m+值域索引)。

练习建议 m≤4、长度≤10 时用回溯分配每个输入位置给某一条序列做核对，随后尝试目标重复值并寻找贪心反例。

原题与解法依据 2025 CCPC 济南 C

第 3 章 二分前缀与滑动窗口

二分将最优化转为判定，前缀状态将区间转为两端之差，差分将范围修改转为边界事件，滑动窗口则复用端点移动后的状态。组合使用前，应先明确下标代表元素位置、操作起点还是前缀状态。

3.1 整数二分与二分答案

二分答案首先需要证明判定的单调性。求最小可行值时，check(x) 从假变真；求最大可行值时，方向相反。搜索区间应覆盖答案，返回值的含义也应明确为可行点或分界旁的不可行点。

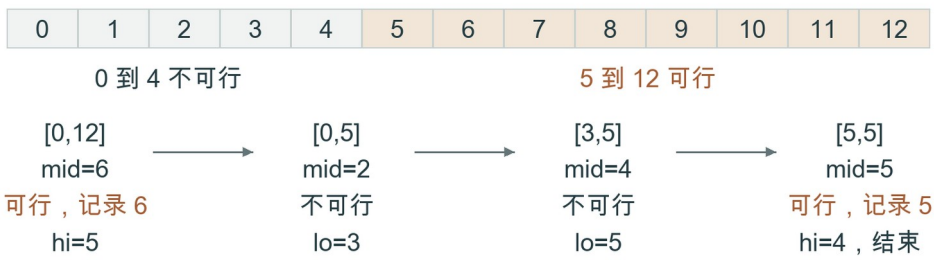


图 3.1 在闭区间中定位第一个可行值

复杂度与接口 O(log(hi−lo+1) × 判定复杂度)。

```
template <class F> optional<ll> first_true(ll lo, ll hi, F check) {
    optional<ll> ans;
    while (lo <= hi) {
        ll mid = (ll)((i128)lo + ((i128)hi - lo) / 2);
        if (check(mid)) {
            ans = mid;
            if (mid == LLONG_MIN)
                break;
            hi = mid - 1;
        } else {
            if (mid == LLONG_MAX)
                break;
            lo = mid + 1;
        }
    }
    return ans;
}
```

实现要点 接口允许整个区间均不可行，此时返回空 optional。中点在 i128 中计算，避免上下界之差溢出；两端不要求预先为可行点。

浮点二分通常采用固定迭代次数。三分要求目标函数单峰；峰值二分则依赖局部方向能够保证目标仍在保留区间内，不能仅凭样例推测。

常见判定模型包括连续分段最小化最大和、第 k 小数对距离、电池分配、服务等待、刀砍与毒伤以及 MEX 分段。非负分段可贪心统计所需段数；数对距离可在排序后用双指针累计距离不超过阈值的数对。

3.2 前缀状态与子数组计数

子数组 [l,r) 的和为 pre[r]−pre[l]，因此统计和为 k 的子数组变成统计此前有多少 pre[l]=pre[r]−k。扫描时先查询再加入当前前缀，初始加入 pre[0]=0，便能统一包含从数组起点开始的答案。

复杂度与接口 哈希期望 O(n)，有序 map 为 O(n log n)；空间 O(n)。

```
ll count_sum(const vector<ll> &a, ll k) {
    unordered_map<ll, ll> cnt;
    cnt.reserve(a.size() * 2 + 1);
    cnt[0] = 1;
    ll pre = 0, ans = 0;
    for (ll x : a) {
        pre += x;
        auto it = cnt.find(pre - k);
        if (it != cnt.end())
            ans += it->second;
        ++cnt[pre];
    }
    return ans;
}
```

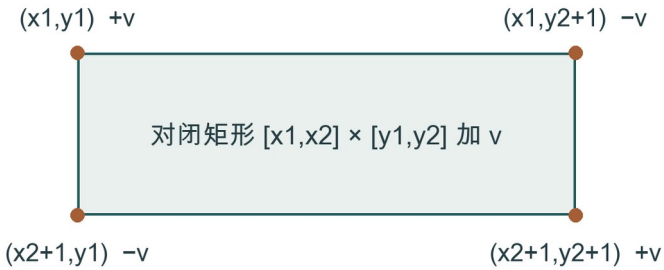
实现要点 先放入空前缀 0，先查询再插入。负数余数要规范为非负值；元素可能为负时不能用“和大了就缩左边”。

把前缀和替换为前缀异或、模余数或奇偶向量，同样可以统计相等或互补状态。键保存状态，频次保存贡献次数；即使元素为负，前缀状态法仍成立。

最长正和子数组可用前缀最小值单调栈配合从右向左扫描。最长正负数量相等子数组先将元素符号转成 ±1。元音出现偶数次用 5 位掩码，相同前缀掩码之间的区间合法。删除最短子数组使剩余和被 p 整除则查 pre−total 模 p 的最后出现位置。

3.3 二维前缀和与矩形差分

二维前缀把左上矩形和缓存起来，任意矩形用两个加项和两个减项恢复。二维差分与它方向相反：给矩形四角写入带符号事件，再沿两维前缀恢复。二者分别压缩查询与修改。



四个边界事件经两维前缀和恢复为矩形增量。

图 3.2 矩形差分的四个边界事件

复杂度与接口 预处理 O(nm)，单次查询或矩形标记 O(1)；离散化 O(n log n)。

```
struct Prefix2D {
    vector<vector<ll>> s;
    explicit Prefix2D(const vector<vector<ll>> &a) {
        int n = a.size(), m = n ? a[0].size() : 0;
        s.assign(n + 1, vector<ll>(m + 1));
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++)
                s[i][j] = a[i - 1][j - 1] + s[i - 1][j] + s[i][j - 1] -
                          s[i - 1][j - 1];
    }
    // 输入为从 1 开始的闭矩形。
    ll sum(int x1, int y1, int x2, int y2) const {
        return s[x2][y2] - s[x1 - 1][y2] - s[x2][y1 - 1] + s[x1 - 1][y1 - 1];
    }
};
void rect_add(vector<vector<ll>> &d, int x1, int y1, int x2, int y2, ll v) {
    d[x1][y1] += v;
    d[x2 + 1][y1] -= v;
    d[x1][y2 + 1] -= v;
    d[x2 + 1][y2 + 1] += v;
}
```

实现要点 压缩后的相邻秩不代表原坐标距离为 1。面积、长度计算必须回到原坐标；矩形差分需要为 r2+1、c2+1 留边界。

原始坐标很大时先分析离散化能否保留长度。计数网格只需排名，面积和距离问题必须保存相邻真实坐标差；压成相邻整数会改变几何量。

3.4 等差数列二阶差分

等差序列的一阶差分为常数，二阶差分在区间内部为零，因此一次等差区间加只需记录四个边界事件：l、l+1、r+1、r+2。对差分数组连续求两次前缀和即可还原。

复杂度与接口 每次修改 O(1)，整体还原 O(n)。

```
void add_ap(vector<ll> &dd, int l, int r, ll s, ll d) {
    ll e = s + (r - l) * d;
    dd[l] += s;
    dd[l + 1] += d - s;
    dd[r + 1] -= e + d;
    dd[r + 2] += e;
```

```
}
vector<ll> restore_ap(vector<ll> dd, int n) {
    for (int k = 0; k < 2; k++)
        for (int i = 1; i <= n; i++)
            dd[i] += dd[i - 1];
    return vector<ll>(dd.begin() + 1, dd.begin() + n + 1);
}
```

实现要点 e 必须等于 s+(r−l)d。r=l 时四处修改有重合，本节的累加写法仍成立。数组至少开 n+3。

先手算长度 1、2 的区间并恢复，再推广到任意长度。问的是若干次操作后的总数组时可离线恢复；若每次修改后立刻查询，需要把差分事件放进动态数据结构。

3.5 双指针与至多 k 类计数

当区间扩大使“种类不超过 k”只能从满足变成不满足，右端前进时左端也只需前进。固定右端 r，合法左端形成连续后缀，因此合法区间数是 r−l+1。恰好 k 类可用 atMost(k) −atMost(k−1)。

复杂度与接口 平均 O(n)，空间与窗口内不同元素数相关。

```
ll at_most(const vector<int> &a, int k) {
    if (k < 0)
        return 0;
    unordered_map<int, int> cnt;
    int l = 0, kinds = 0;
    ll ans = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        if (++cnt[a[r]] == 1)
            ++kinds;
        while (kinds > k) {
            if (--cnt[a[l]] == 0) {
                cnt.erase(a[l]);
                --kinds;
            }
            ++l;
        }
        ans += r - l + 1;
    }
    return ans;
}
```

实现要点 atMost(−1) 应为 0。区间约束是否对缩小窗口封闭，决定能否双指针。最短覆盖串要维护缺少的字符数，不能只维护不同字符数。

双指针的线性界来自每个端点最多移动 n 次。区间和约束只有在元素非负等条件下才有相同单调性；负数数组应改用前缀状态或单调队列。

窗口还能维护最小覆盖串、无重复字符最长串与平衡串替换。每字符至少 k 次可枚举目标字符种类数，使窗口重新获得单调约束。环形区间可复制一段数组；两端接雨水、救生艇和最大容器则通过局部上界判断该移动哪一侧。供暖半径可排序后双指针或二分，重复数可用快慢指针，最小缺失正数可用原地置换。它们共享移动与状态维护思想，单调性理由各不相同。

3.6 可删除的中位数与绝对差代价

绝对差之和在中位数处最小。用两个有序多重集合，将较小的一半放 lo，较大的一半放 hi；保证 lo 大小等于 hi 或多一，并保存两边的元素和。这样代价可由中位数乘数量减元素和得到。

复杂度与接口 插入删除 O(log n)，查询代价 O(1)，空间 O(n)。

```
struct Median {
    multiset<ll> L, R;
    ll sl = 0, sr = 0;
    void balance() {
        while (L.size() > R.size() + 1) {
            auto it = prev(L.end());
            ll x = *it;
            L.erase(it);
            sl -= x;
            R.insert(x);
            sr += x;
        }
        while (L.size() < R.size()) {
            auto it = R.begin();
            ll x = *it;
            R.erase(it);
            sr -= x;
            L.insert(x);
            sl += x;
        }
    }
    void add(ll x) {
        if (L.empty() || x <= *L.rbegin())
            L.insert(x), sl += x;
        else
            R.insert(x), sr += x;
        balance();
    }
    void erase(ll x) {
        auto it = L.find(x);
        if (it != L.end())
            L.erase(it), sl -= x;
        else {
            it = R.find(x);
            assert(it != R.end());
            R.erase(it);
            sr -= x;
        }
        balance();
    }
    ll cost() const {
        if (L.empty())
            return 0;
        ll m = *L.rbegin();
        return m * (ll)L.size() - sl + sr - m * (ll)R.size();
    }
};
```

实现要点 删除一个重复值时要 erase(iterator)，不能 erase(value)。每次增删后重新平衡；空窗口的代价为 0。

只插入的中位数可用双堆维护；滑动窗口还需处理任意位置删除，可选择多重集合、延迟删除或频次树状数组。

3.7 典型题解析

例题 3.1 Flipping Cards 中位数与最大子段和

2023 CCPC 桂林 M

问题 n 张牌两面分别为 a[i] 与 b[i]，可翻一个连续区间，也可不翻，求正面中位数的最大值。

解法 试探中位数阈值 x。先按原题中位数的定义，计算至少需要多少张正面牌达到 x。

原有达标数量是 base=Σ[a[i]≥x]。翻第 i 张的增益是 [b[i]≥x]−[a[i]≥x]，只有 −1、0、1。

翻区间的总增益是该增益数组的子段和，因此 check(x) 用允许空区间的 Kadane 计算最大增益。

base 加最大增益达到要求即为可行；阈值增大不会产生新可行解，二分求最大值。

正确性 每种翻区间方案在给定阈值下的达标数，恰等于 base 加区间增益。最大子段和枚举了所有合法连续翻转，因此判定无遗漏。降低阈值会让同一翻转方案的达标数不减少，故判定单调。

示例推演 a=[1,5,2,4,3]，b=[6,1,6,1,6]。试 x=4，base=2，增益为 [1,−1,1,−1,1]，最大增益是 1，至少 3 张达标，因此可行。x=5 同样可行；x=6 只有第 1、3、5 张达标，翻整段即可达到 3 张。

```
bool cards_check(const vector<ll> &a, const vector<ll> &b, ll x, int need) {
    int base = 0, end = 0, best = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        base += a[i] >= x;
        int gain = (b[i] >= x) - (a[i] >= x);
        end = max(0, end + gain);
        best = max(best, end);
    }
    return base + best >= need;
}
```

实现要点 不要对每个试探阈值排序两面数据，排序破坏连续性。允许不翻时最大增益初值必须为 0。偶数长度的中位数必须按题目是下中位数还是上中位数确定 need。

复杂度 O(n log V)，check O(n)，额外空间 O(1)。

练习建议 先写 $O(n^2)$ 翻区间暴力；n≤8 时枚举所有小整数两面牌，用暴力答案核对二分。随后练 2022 南京 D。

原题与解法依据 2023 CCPC 桂林 M

例题 3.2 Chat Program 二分后的差分扫描

2022 ICPC 南京 D

问题 对长度为 m 的连续段至多加一次等差序列，首项 c、公差 d，最大化整个数组第 k 大的值。

解法 试阈值 x，把每个原数是否≥x 转成 0 或 1，目标是至少有 k 个 1。

固定元素位置 t，枚举操作起点 s 的合法范围 $\max(1,t-m+1)\le s\le \min(t,n-m+1)$。

该元素受到的加值是 $c+(t-s)d$，满足 $a[t]+c+(t-s)d\ge x$ 的起点形成一个区间。按 d 的符号和 d=0 分别求区间。

对这个起点区间做差分加一，扫出每个操作起点的达标数量。原本已达标元素如果修改会使它失效，也必须计入增益或损失。

正确性 一个元素对不同操作起点的变化是关于 s 的一次函数，因此过阈值的起点是连续区间。把所有元素的影响叠加以后，每个起点的总达标数就是准确判定。

示例推演 若 m=3、c=2、d=1，位置 t=4 的元素在起点 s=2、3、4 中分别增加 4、3、2。假设 a[4]=5、x=8，满足条件的起点为 [2,3]，只需 diff[2]++、diff[4]--。

实现要点 不能把负数除法当成数学向下取整。先核对原题 c、d 范围；如加值恒非负，原本达标者可直接进入 base。如允许负加值，要用完整“新达标减旧达标”差分。

复杂度 $O(n \log A)$，每次判定 $O(n)$，空间 $O(n)$。

练习建议 练习手算每个元素的有效起点区间，并对 d=0、正数、负数写小数据暴力核对。将整数 ceil_div 和 floor_div 单独验证。

原题与解法依据 2022 ICPC 南京 D

第 4 章 区间数据结构与离线方法

数据结构维护的是状态摘要。先确定节点需要保存什么、修改如何作用于摘要，再选择树状数组、线段树、单调结构或有序集合；询问已知时，还可通过离线排序简化维护。

4.1 单调栈与贡献边界

贡献法以每个位置作为区间最小值计数。找到它左侧与右侧最近的阻挡边界，左端可选数乘右端可选数就是它的贡献次数。重复值必须由一侧严格、另一侧非严格来划分归属，否则同一段会重复或遗漏。

复杂度与接口 时间 O(n)，空间 O(n)。

```
i128 sum_min(const vector<ll> &a) {
    int n = a.size();
    vector<int> L(n), R(n), st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.back()] >= a[i])
            st.pop_back();
        L[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }
    st.clear();
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && a[st.back()] > a[i])
            st.pop_back();
        R[i] = st.empty() ? n : st.back();
        st.push_back(i);
    }
    i128 ans = 0;
    for (int i = 0; i < n; i++)
        ans += (i128)a[i] * (i - L[i]) * (R[i] - i);
    return ans;
}
```

实现要点 重复值必须固定归属；如果两边都严格更小，会重复计数；两边都小于等于会漏计。乘法采用 i128。

例如 [2,2] 共有三个子数组，两个位置不能都认为长度二的子数组归自己。把相等值固定归给最左或最右位置，是证明的一部分，不是写法偏好。

最大宽度坡使用前缀最小下标栈，右端点倒序扫描；去重字母维护剩余次数，只有未来还能补回的字母才允许弹出；大鱼吃小鱼维护删除轮数；全 1 子矩阵先把每行变为柱高，再计算以当前行作底的矩形数量。这些是同一类“弹出时确定职责”的应用。

4.2 单调队列与滑动极值

单调队列维护的是未来仍可能成为最优的候选。新元素如果值更好、过期更晚，就完全支配旧元素，旧元素可以从尾部删除；队首负责删除已过期候选。

复杂度与接口 总时间 O(n)；定长极值空间 O(k)，最短子数组版空间 O(n)。

```
vector<ll> window_max(const vector<ll> &a, int k) {
    assert(k >= 1 && k <= (int)a.size());
    deque<int> q;
    vector<ll> ans;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!q.empty() && q.front() <= i - k)
            q.pop_front();
        while (!q.empty() && a[q.back()] <= a[i])
            q.pop_back();
        q.push_back(i);
        if (i >= k - 1)
            ans.push_back(a[q.front()]);
    }
    return ans;
}
int shortest_at_least(const vector<ll> &a, ll k) {
    int n = a.size(), ans = n + 1;
    vector<ll> s(n + 1);
    deque<int> q;
    for (int i = 0; i < n; i++)
        s[i + 1] = s[i] + a[i];
    for (int r = 0; r <= n; r++) {
        while (!q.empty() && s[r] - s[q.front()] >= k)
            ans = min(ans, r - q.front()), q.pop_front();
        while (!q.empty() && s[q.back()] >= s[r])
            q.pop_back();
        q.push_back(r);
    }
    return ans == n + 1 ? -1 : ans;
}
```

实现要点 队列保存下标才能判过期。负数数组的最短和≥k 子数组用前缀和的单调队列，不能套普通窗口。

同一候选维护思想也适用于 dp[i]=a[i]+min(dp[j])：若 j 的合法区间单调移动，且较优候选能持续支配较差候选，就可以用单调队列维护转移。

最大最小值之差受限的窗口需要两条单调队列。坐标跨度受限的最大表达式可把与当前点无关的部分放入队列；任务分配常先二分任务数，再用双端队列贪心检查。窗口 DP 将候选值换成 dp[j]，第 5 章的 Ropeway 展示这一转换。

4.3 树状数组与动态前缀

Fenwick 的每个节点保存长度 lowbit(i) 的一段后缀。查询前缀时向前删去这些二进制块；修改一个位置时向后找到所有覆盖它的块。每次循环至少处理一位，因此为对数复杂度。

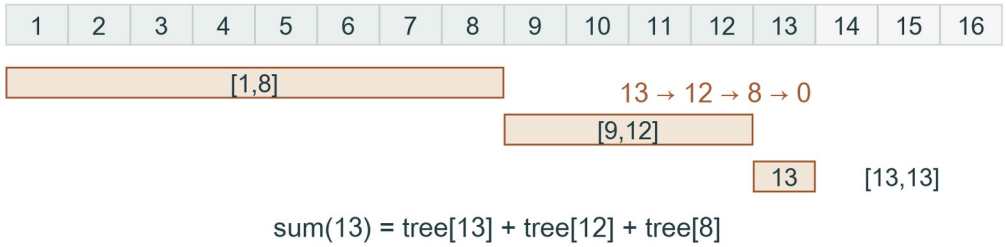


图 4.1 前缀 13 的树状数组块分解

复杂度与接口 修改与查询 O(log n)，空间 O(n)。

```
struct Fenwick {
    int n;
    vector<ll> bit;
    explicit Fenwick(int n) : n(n), bit(n + 1) {}
    void add(int i, ll v) {
        assert(1 <= i && i <= n);
        for (; i <= n; i += i & -i)
            bit[i] += v;
    }
    ll sum(int i) const {
        ll s = 0;
        for (; i > 0; i -= i & -i)
            s += bit[i];
        return s;
    }
    ll range(int l, int r) const { return sum(r) - sum(l - 1); }
    int kth(ll k) const {
        assert(k >= 1 && k <= sum(n));
        int x = 0, step = 1;
        while (step <= n / 2)
            step <<= 1;
        for (; step; step >>= 1)
            if (x + step <= n && bit[x + step] < k)
                k -= bit[x += step];
        return x + 1;
    }
};
```

实现要点 add(0,…) 会死循环。kth 仅适用于每个位置频次非负、且 1≤k≤总频次；模意义的阶乘进制排名不能拿来还原精确排列。

用在逆序对时，树中存已扫元素的频次；用在不同数离线查询时，树中存各值最后出现位置上的标记。数据结构不变，真正需要训练的是“节点里应该存什么”。

4.4 树状数组的区间修改与第 k 小查询

前缀和不是树状数组的唯一用途。对差分数组建树，可以把区间加拆成两个单点；若存储非负频次，还能在树的隐式层级上直接寻找第 k 个元素。

设 d 为原数组差分。前 x 项和等于 $\Sigma_{(i\le x)}(x-i+1)d[i]$，整理为 $(x+1)\Sigma d[i]-\Sigma i\cdot d[i]$。因此维护 d[i] 和 i·d[i] 两棵树即可。寻找第 k 个元素时，从最高二进制位开始试探前缀位置：若整块频次仍小于 k，就跳过它并减去其贡献。

正确性 区间加在 l 放入 v，在 r+1 放入 −v；差分恢复后恰好只改变 [l,r]。倍增查找每次保留“当前前缀和小于原目标”的不变量，最后的下一个位置就是首次达到 k 的位置。前缀单调性依赖每个位置的频次非负。

复杂度与接口 区间加、区间和与第 k 小均为 O(log n)；两棵树占 O(n) 空间。依赖 Fenwick 的 add 与 sum 接口，位置从 1 开始。

```
struct RangeBIT {
    int n;
    Fenwick a, b;
    RangeBIT(int n) : n(n), a(n), b(n) {}
    void range_add(int l, int r, ll v) {
        a.add(l, v);
        b.add(l, v * l);
        if (r < n) {
            a.add(r + 1, -v);
            b.add(r + 1, -v * (r + 1));
        }
    }
    ll prefix(int x) { return (x + 1) * a.sum(x) - b.sum(x); }
    ll query(int l, int r) { return prefix(r) - prefix(l - 1); }
};
// 第 k 小直接使用上一节 Fenwick 的 kth(k)。
```

实现要点 k 必须在 1 到总频次之间。删除后出现负频次会破坏倍增查找。区间加的 v·i 和累计和可能溢出，需要根据题目提升整数类型。

先维护多重集合的插入、删除和第 k 小，再把它用于窗口中位数。若需任意实数键，先离散化。

4.5 单点修改与区间最大值

线段树通过区间二分把任意查询分解成 O(log n) 个节点区间。每个节点保存可合并的摘要，最大值的合并是 max，区间和的合并是加法。节点摘要必须足够回答上层查询。

复杂度与接口 初始化 O(n)，更新与查询 O(log n)，空间 O(n)。

```
struct SegMax {
    int n;
    vector<ll> t;
    explicit SegMax(const vector<ll> &a) : n(a.size()), t(2 * n, -INF) {
        for (int i = 0; i < n; i++)
            t[n + i] = a[i];
        for (int i = n - 1; i; i--)
            t[i] = max(t[i << 1], t[i << 1 | 1]);
    }
    void set(int p, ll v) {
        for (t[p += n] = v; p > 1; p >>= 1)
            t[p >> 1] = max(t[p], t[p ^ 1]);
    }
    ll query(int l, int r) const {
        ll ans = -INF;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1)
                ans = max(ans, t[l++]);
            if (r & 1)
```

```
                ans = max(ans, t[--r]);
        }
        return ans;
    }
};
```

实现要点 最大值的单位元应为负无穷，不能用 0 处理可能为负的数组。求和、计数、第 k 个等操作需要更换合并规则和单位元。

最大子段和查询需同时保存区间总和、最大前缀和、最大后缀和与最大子段和。节点信息应由合并公式反推，单独保存 sum 不足以完成这种查询。

4.6 区间加与区间和的懒标记线段树

区间加 v 时，节点总和立即增加 v×区间长度，同时将 v 累加到懒标记。标记表示尚未同步给孩子的统一增量；只有继续访问孩子时才需下传。

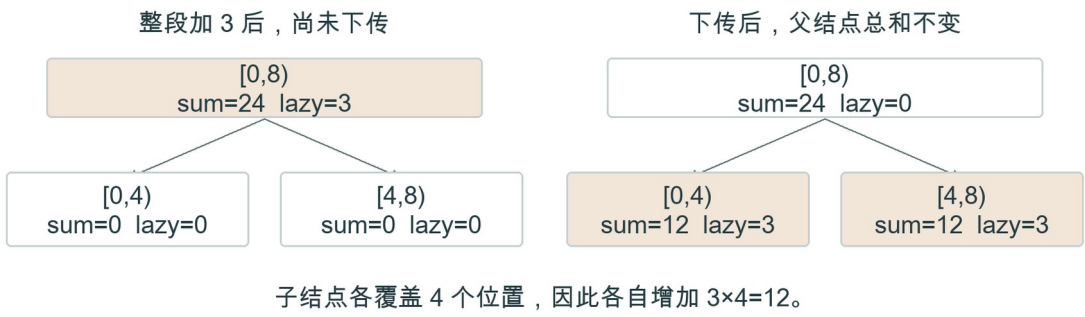


图 4.2 区间增量的记录与下传

复杂度与接口 建树 O(n)，修改查询 O(log n)，空间 O(n)。

```
struct LazySum {
    int n;
    vector<ll> s, z;
    explicit LazySum(const vector<ll> &a) : n(a.size()), s(4 * n), z(4 * n) {
        assert(n);
        build(1, 0, n, a);
    }
    void build(int u, int l, int r, const vector<ll> &a) {
        if (r - l == 1) {
            s[u] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(u * 2, l, m, a);
        build(u * 2 + 1, m, r, a);
        s[u] = s[u * 2] + s[u * 2 + 1];
    }
    void apply(int u, int len, ll v) {
        s[u] += len * v;
        z[u] += v;
    }
    void push(int u, int l, int r) {
        int m = (l + r) / 2;
```

```
        apply(u * 2, m - l, z[u]);
        apply(u * 2 + 1, r - m, z[u]);
        z[u] = 0;
    }
    void add(int ql, int qr, ll v, int u, int l, int r) {
        if (qr <= l || r <= ql)
            return;
        if (ql <= l && r <= qr) {
            apply(u, r - l, v);
            return;
        }
        push(u, l, r);
        int m = (l + r) / 2;
        add(ql, qr, v, u * 2, l, m);
        add(ql, qr, v, u * 2 + 1, m, r);
        s[u] = s[u * 2] + s[u * 2 + 1];
    }
    ll sum(int ql, int qr, int u, int l, int r) {
        if (qr <= l || r <= ql)
            return 0;
        if (ql <= l && r <= qr)
            return s[u];
        push(u, l, r);
        int m = (l + r) / 2;
        return sum(ql, qr, u * 2, l, m) + sum(ql, qr, u * 2 + 1, m, r);
    }
    void add(int l, int r, ll v) { add(l, r, v, 1, 0, n); }
    ll sum(int l, int r) { return sum(l, r, 1, 0, n); }
};
```

实现要点 赋值、加法、仿射变换的标记组合不同；本实现仅支持加法。所有接口都是从 0 开始的半开区间。

若同时支持乘和加，把作用表示为仿射函数 $x \mapsto ax+b$，复合顺序不可交换。赋值标记同样不能与加法随意互换，要按操作时间合成。

4.7 静态 RMQ 的 Sparse Table

ST 预处理长度为 2^k 的区间摘要。查询取两个可能重叠的等长区间，因此 O(1) 只适用于 min、max、gcd 等幂等操作。区间和的两段会把重叠部分算两遍。

复杂度与接口 预处理 O(n log n)，查询 O(1)，空间 O(n log n)。

```
struct SparseMin {
    vector<int> lg;
    vector<vector<ll>> st;
    explicit SparseMin(const vector<ll> &a) : lg(a.size() + 1) {
        int n = a.size();
        assert(n);
        for (int i = 2; i <= n; i++)
            lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, vector<ll>(n));
        st[0] = a;
        for (int k = 1; k <= lg[n]; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
```

```
    ll query(int l, int r) const {
        int k = lg[r - l];
        return min(st[k][l], st[k][r - (1 << k)]);
    }
};
```

实现要点 不能把同样的重叠写法用于求和。查询采用 [l,r)，要求 l<r；静态数组修改后表失效。

Sparse Table 适用于静态数组。单点修改可能影响大量预处理区间；若操作序列包含修改，应重新考虑树状数组或线段树。

4.8 离线查询与普通莫队

当所有询问提前给出，且区间答案可以用一次加入或删除快速更新时，可以改变询问次序，复用相邻区间的状态。莫队并不要求答案可合并，它要求端点移动可维护。

把左端点按长度 B 分块，块内按右端点排序；相邻块交替使用升序和降序，减少右端点来回扫描。维护当前闭区间 [L,R]，每一步只加或删一个位置。不同数个数的更新只关心该值的频次是否跨过零。

正确性 每次移动后，频次数组准确描述当前区间。一个左端块内右端累计移动 O(n)，共有 O(n/B) 块；左端移动合计 O(qB)。取 B≈n/√q，可得到 O(n√q+q) 量级的端点移动，常用 B≈√n 是 q≈n 时的选择。

复杂度与接口 排序 O(q log q)，移动 O(n²/B+qB)，空间 O(n+q+值域)。本实现仅处理静态不同数询问，修改莫队另需时间维。

```
struct Query {
    int l, r, id;
};
vector<int> distinct_mo(const vector<int> &a, vector<Query> qs, int kinds) {
    int n = a.size();
    int B = max(1, int(n / sqrt(max(1, int(qs.size())))));
    sort(qs.begin(), qs.end(), [&](Query x, Query y) {
        int bx = x.l / B, by = y.l / B;
        if (bx != by)
            return bx < by;
        return (bx & 1) ? x.r > y.r : x.r < y.r;
    });
    vector<int> cnt(kinds), ans(qs.size());
    int L = 0, R = -1, cur = 0;
    auto add = [&](int p) {
        if (cnt[a[p]]++ == 0)
            ++cur;
    };
    auto del = [&](int p) {
        if (--cnt[a[p]] == 0)
            --cur;
    };
    for (auto q : qs) {
        while (L > q.l)
            add(--L);
        while (R < q.r)
            add(++R);
```

```
        while (L < q.l)
            del(L++);
        while (R > q.r)
            del(R--);
        ans[q.id] = cur;
    }
    return ans;
}
```

实现要点 下标统一为 0 开始，id 必须覆盖 0..q−1。先扩张再收缩可避免从空区间删除。存在单调扫描的专门解法时，应比较它与莫队：静态不同数可以用最后出现位置加 Fenwick 做到 $O((n+q)\log n)$。

4.9 结构选型与摊还分析

需求	首选工具	成立条件
静态区间和	前缀和	数据不修改
单点加与区间和	Fenwick	可加减的前缀摘要
静态区间最值	ST	幂等合并与静态数据
区间加与区间和	懒标记线段树	摘要能接受整段作用
窗口最值	单调队列	候选支配与单调过期
动态前后继	set 或 multiset	有序键与重复值语义

摊还分析考察整个操作序列的总成本。单调栈中每个元素至多入栈、出栈一次，因而总时间为线性；若允许撤销并重新引入已删除元素，则需重新证明这一界限。

链表操作同样遵循局部关系维护：反转保存后继指针，快慢指针定位中点或环入口，分组反转先确认组长。随机指针复制可用映射或穿插副本；LRU 组合哈希表与双向链表，频率栈按频次分组，随机集合通过尾元素换位实现删除。

4.10 典型题解析

例题 4.1 滑动窗口的最小值与最大值

洛谷 P1886 基础验证

问题 固定长度 k 的窗口在数组上逐格右移，输出每个窗口的最小值与最大值。

解法 队列存下标，不存值。求最小值时，新元素进入前删去尾部所有不小于它的元素，再删去队首已经离开窗口的下标。队首就是答案；最大值反向比较即可。

正确性与复杂度 被新元素淘汰的旧元素值更差，且旧元素更早离开窗口，因此以后永远不会优于新元素。每个下标入队、出队各一次，整个过程 $O(n)$。

示例推演 数组 [4,2,2,5]、k=3，两个最小值均为 2。若用 ≥ 删除尾部，第二个 2 替代第一个 2，它更晚过期，答案仍正确。

实现要点 输出从 i=k−1 开始。不同数统计等窗口问题不能直接把单调队列当作频次结构。

练习建议 先输出队列下标轨迹，再将 min 转移改写成窗口 DP。

原题与解法依据 洛谷 P1886 基础验证

例题 4.2 动态排名的离散化解法

洛谷 P3369 基础验证

问题 维护可重集合，支持插入、删一个副本、排名、第 k 小、严格前驱与严格后继。操作全部给出。

解法 预读并离散化所有可能插入的值。Fenwick 记录各排名频次；小于 x 的数量由 lower_bound 位置之前的前缀和给出，排名加一。第 k 小使用频次倍增；前驱与后继转成对应频次序号，再反查真实值。

正确性与复杂度 离散化保持大小次序与重复值身份，前缀频次就是排名定义。倍增查找首次累计达到 k 的位置，找到排序后的第 k 个元素。总时间 O(q log q)。

示例推演 集合 [2,2,7] 中 7 的排名为 3，第二小为 2。查询 5 的严格前驱是 2，后继是 7，即使 5 不在集合里也可用二分定位。

实现要点 删除一个副本只减一。前驱不能用 ⩽x 的计数，后继不能用 ⩾x 的起点，否则集合含 x 时会错误返回 x 自己。

练习建议 将全部六种操作与有序数组模拟比较，特别测试大量重复值。

原题与解法依据 洛谷 P3369 基础验证

例题 4.3 Barkley II MEX 分段与离线不同数查询

2023 CCPC 桂林 I

问题 选择一个连续区间，使其中不同值数量减去 MEX 最大。MEX 为最小没出现的非负整数。

解法 枚举候选 x，把所有 x 出现的位置当成切点，只考虑不含 x 的极长区间。

一个区间不含 x，故真实 MEX≤x。以“不同值数−x”更新答案不会高估。真实最优区间的 MEX=t 时，在 x=t 这一轮扩展到极长区间不会损失。

收集所有候选区间，按右端点排序，用 Fenwick 维护每个数最后出现位置上的 1。处理 a[r] 时撤掉旧 last[a[r]]，在 r 放 1。

区间 [l,r] 的不同值数就是 Fenwick.sum(r)−Fenwick.sum(l−1)。MEX 不超过 n，只需枚举 0..n。

正确性 任意候选得分 D−x≤D−真实 MEX，所以不会产生虚假上界。取真实最优 MEX 的候选时，极长扩展增加或保持不同值数，且仍缺失该 MEX，能够覆盖最优解。查询数等于每个值出现次数加一个，合计 O(n)。

示例推演 a=[0,2,2,3]。当 x=1 时整段不含 1，不同值有 3 类，得分 2。扫描到 r=4 时，只在下标 1、3、4 放 1，查询任意左端点都可正确处理重复的 2。

实现要点 不要枚举所有 $O(n^2)$ 区间。题面值域大也不需要枚举到最大 a[i]；只记录$\leqslant$n 的 MEX 切点，扫描时所有值仍参与不同值计数。

复杂度 O(n log n) 时间，O(n) 个候选查询，空间 O(n)。

练习建议 先做静态不同数区间查询，再实现生成候选区间；对所有 n≤8 数组暴力核对。

原题与解法依据 2023 CCPC 桂林 I

例题 4.4 Trapping Rain Water 从双指针到动态单调栈

2023 ICPC 南京 M

问题 柱高数组中每次仅增加一个位置的高度，操作后求可接雨水总量。

解法 静态水位是 min(前缀最大,后缀最大)。对每个位置，两边最大值中至少一边等于全局最大 M，因此水位也等于前缀最大+后缀最大−M。

总水量等于前缀最大之和+后缀最大之和−nM−柱高总和。现在只需动态维护两套前后缀纪录高。

用 set 保存每段前缀最大开始的下标。提高 p 后，若超过旧前缀最大，就截断 p 所在段，并删除右侧被新高度压过的纪录。后缀同理反向处理。

维护每个常值段的高度乘长度之和。一次可能删除很多节点，但每次修改最多加入一个新纪录。

正确性 前缀最大从左向右不降，一旦某条纪录被更高的后起位置淘汰，它不会在之后仅增高的修改中重新产生。新增纪录总数 O(n+q)，所以所有删除总数同阶。

示例推演 柱高 [3,0,2,0,4] 的水量为 7。把中间高度 2 提到 5 后，前缀最大变为 [3,3,5,5,5]，后缀最大为 [5,5,5,4,4]，用总和公式得到 7；不能只重算被修改的一格。

实现要点 只增不减是摊还证明的核心，允许降低高度后上述删除策略失效。修改要更新柱高总和、全局最大以及两个方向的段边界。

复杂度 O((n+q) log(n+q)) 总时间，空间 O(n+q)。

练习建议 先练“单点只增后的前缀最大之和”，用 O(n) 重算做对拍，再加入后缀。

原题与解法依据 2023 ICPC 南京 M

例题 4.5 Activity Rehearsal 用依赖树压缩队列模拟

2025 CCPC 济南 L

问题 排列入队，每次取队头两项，较大者输出，较小者放队尾，共 n−1 次。支持交换原数组两项，以及查询第 l..r 次输出之和。

解法 把队列过程展开成一个延长数组 a[1..2n−1]，第 i 次操作产生 a[n+i]=min(a[2i−1],a[2i])，输出 b[i]=max(这两项)。

由新值 n+i 指向两个来源 2i−1、2i，得到一棵依赖二叉树。编号变化后相当于完全二叉树层次，深度 O(log n)。

保存每个内部点的较小值，并用 Fenwick 保存对应 b[i]。改一个原数组叶子时，沿唯一祖先链重算；只对输出改变的项做差值修改。

交换两项可以顺序改两片叶子，最后状态正确。查询用输出数组的区间和。

正确性 每个延长数组位置只会成为一次后续操作的输入，因此父亲唯一。用 j=2n−编号换元后，父子编号约为 j 与 2j、2j+1，深度是对数级。叶子修改只影响祖先，全部其余输出不变。

示例推演 n=4、a=[3,1,4,2]。输出依次是 3、4、2，延长项依次为 1、2、1。依赖关系是节点 5←(1,2)，6←(3,4)，7←(5,6)。交换下标 1 与 4 后，分别沿两条祖先链重算。

实现要点 原始位置和第几次操作是两套编号。Fenwick 更新的是 b[i] 的差，不能写成新值累加。别复制队列跑完整 n 次来回答每个询问。

复杂度 建结构 O(n)，一次交换 O(log² n)，区间和 O(log n)，空间 O(n)。

练习建议 写最直接队列模拟作为真值，随机做交换及查询，对小 n 连续核对；再记录每次实际更新的祖先数。

原题与解法依据 2025 CCPC 济南 L

第 5 章 动态规划与状态压缩

动态规划把对未来影响相同的历史合并为一个状态。状态充分性决定递推是否成立，初始化与依赖顺序决定实现是否正确。本章依次讨论线性、区间、背包、集合和数位状态。

5.1 Kadane 与两个不重叠子段

最大子段和的状态是“必须在当前位置结束”的最优值，转移决定接上前一段还是从当前元素重新开始。最大总答案与结尾状态要分别维护。

复杂度与接口 O(n)，空间单段 O(1)，两段 O(n)。

```
ll kadane(const vector<ll> &a) {
    assert(!a.empty());
    ll end = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); i++)
        end = max(a[i], end + a[i]), best = max(best, end);
    return best;
}
ll two_subarrays(const vector<ll> &a) {
    int n = a.size();
    assert(n >= 2);
    vector<ll> L(n), R(n);
    ll e = a[0];
    L[0] = e;
    for (int i = 1; i < n; i++)
        e = max(a[i], e + a[i]), L[i] = max(L[i - 1], e);
    e = a[n - 1];
    R[n - 1] = e;
    for (int i = n - 2; i >= 0; i--)
        e = max(a[i], e + a[i]), R[i] = max(R[i + 1], e);
    ll ans = -INF;
    for (int i = 0; i < n - 1; i++)
        ans = max(ans, L[i] + R[i + 1]);
    return ans;
}
```

实现要点 全负数组不能初始化答案为 0，除非题目允许空子段。接口要求数组非空。

允许空段时可以把结尾状态截为零；要求非空时不能把全负数组答案变成零。两段不重叠可预处理左侧最优与右侧最优，枚举分界组合。

5.2 LIS 与双序列动态规划

LIS 的 tails[len] 维护长度为 len+1 的递增子序列最小末尾。更小末尾只会给后续元素更多选择，所以可以压缩为一个数。严格递增使用 lower_bound，非降使用 upper_bound。

复杂度与接口 LIS O(n log n)，LCS O(nm)，滚动数组 O(m) 空间。

```
int lis(const vector<ll> &a) {
    vector<ll> t;
    for (ll x : a) {
```

```
        auto it = lower_bound(t.begin(), t.end(), x);
        if (it == t.end())
            t.push_back(x);
        else
            *it = x;
    }
    return t.size();
}
int lcs(const string &a, const string &b) {
    vector<int> d(b.size() + 1);
    for (char x : a) {
        int old = 0;
        for (int j = 1; j <= (int)b.size(); j++) {
            int save = d[j];
            d[j] = x == b[j - 1] ? old + 1 : max(d[j], d[j - 1]);
            old = save;
        }
    }
    return d.back();
}
```

实现要点 严格递增用 lower_bound，非下降用 upper_bound。tails 不是实际子序列；要还原必须记录前驱。滚动 LCS 需要保存左上格旧值。

LCS、编辑距离和交错字符串则同时记两个前缀长度。滚动数组应确认每格读取的是上一行还是本行，遍历方向随依赖关系决定。

5.3 区间 DP 与最长回文子序列

区间 DP 的状态只描述一段连续区间，先算短区间再算长区间。转移可以合并左右端，也可以枚举切分点。最长回文子序列相等端点可一同选入，否则去掉一侧取较优。

复杂度与接口 回文子序列 $O(n^2)$ 时间和空间；扰乱字符串朴素 $O(n^4)$。

```
int lps(const string &s) {
    int n = s.size();
    if (!n)
        return 0;
    vector<vector<int>> d(n, vector<int>(n));
    for (int l = n - 1; l >= 0; l--) {
        d[l][l] = 1;
        for (int r = l + 1; r < n; r++)
            d[l][r] = s[l] == s[r]
                          ? 2 + (l + 1 <= r - 1 ? d[l + 1][r - 1] : 0)
                          : max(d[l + 1][r], d[l][r - 1]);
    }
    return d[0][n - 1];
}
```

实现要点 子序列不连续；回文子串是另一类问题。n=0 要返回 0，不能访问 dp[0][−1]。

矩阵连乘、区间合并费用等枚举切分时通常为 $O(n^3)$。没有四边形不等式等证明，不要擅自把最优分界限制在相邻状态之间。

5.4 0 1 背包与完全背包

压缩为一维后，0 1 背包倒序遍历容量，读取加入当前物品之前的状态；完全背包正序遍历，允许继续读取当前物品刚更新的状态。循环方向由依赖关系决定。



图 5.1 容量遍历方向与当前物品的重复使用

复杂度与接口 O(nW)，空间 O(W)。

```
ll knapsack01(int W, const vector<pair<int, ll>> &items) {
    vector<ll> d(W + 1);
    for (auto [w, v] : items)
        for (int c = W; c >= w; c--)
            d[c] = max(d[c], d[c - w] + v);
    return d[W];
}
ll knapsack_unbounded(int W, const vector<pair<int, ll>> &items) {
    vector<ll> d(W + 1);
    for (auto [w, v] : items) {
        assert(w > 0);
        for (int c = w; c <= W; c++)
            d[c] = max(d[c], d[c - w] + v);
    }
    return d[W];
}
```

实现要点 重量必须为正。恰好装满时 dp[0]=0，其余负无穷；本实现采用至多容量的最大非负收益，初始化为 0。

当容量很大而总价值较小时，可改用“达到某个价值所需的最小重量”作为状态。恰好装满与不超过容量的初始化不同，选维度与初值都应服从题目语义。

5.5 多重背包与分组背包

背包首先要决定物品能用几次以及每组能选几件。多重背包可以把数量拆成二进制包，再使用 0 1 背包；分组背包的转移必须从处理该组之前的状态出发。

数量 s 拆为 1、2、4……和剩余部分，每个包的重量和价值同时乘包内数量。拆出的包可表示 0..s 的任意数量。分组背包中，先保存 old=dp，再让该组所有物品从 old 转移，避免同组多选。恰好装满时初始化 dp[0]=0、其他 −INF；容量不超过时可全部初始化零。

正确性 二进制包的可表示范围逐段连续扩张，覆盖每一种合法取件数。old 隔离上一组状态，使每次新增选择恰好来自当前组的一个物品；允许不选时保留旧 dp。

复杂度与接口 二进制拆分 O(WΣlog(s_i+1))，分组背包 O(W×总物品数)，空间 O(W)。重量须为正；无限零重量正价值物品会使最优值无界。

```
void bounded_item(vector<ll> &dp, int w, ll v, int s) {
    int W = int(dp.size()) - 1;
    for (ll k = 1; s > 0; k *= 2) {
        int take = int(min<ll>(k, s));
        s -= take;
        ll weight = 1LL * take * w, value = take * v;
        if (weight > W)
            continue;
        for (int c = W; c >= weight; --c)
            dp[c] = max(dp[c], dp[c - int(weight)] + value);
    }
}
void group_items(vector<ll> &dp, const vector<pair<int, ll>> &group) {
    auto old = dp;
    int W = int(dp.size()) - 1;
    for (auto [w, v] : group)
        for (int c = w; c <= W; ++c)
            dp[c] = max(dp[c], old[c - w] + v);
}
```

实现要点 模板使用“容量不超过”语义。恰好装满或负价值时，应对不可达状态做保护，不能让 −INF 加上价值伪装成可达。计数问题还要分清组合与排列：物品在外通常数无序组合，容量在外可能数有序序列。

5.6 集合状压 DP 的基础骨架

集合状压先证明未来只与当前 mask 有关。分配问题中已选对象数量 popcount(mask) 决定下一位，无需再存阶段；旅行路线还依赖当前位置，因此必须增加 last。

复杂度与接口 $O(n2^n)$，空间 $O(2^n)$，一般 $n \le 20$。

```
ll assignment(const vector<vector<ll>> &cost) {
    int n = cost.size();
    assert(n <= 20);
    int S = 1 << n;
    vector<ll> d(S, INF);
    d[0] = 0;
    for (int mask = 0; mask < S; mask++) {
        int i = __builtin_popcount((unsigned)mask);
        if (i == n)
            continue;
        for (int j = 0; j < n; j++)
            if (!(mask >> j & 1))
                d[mask | 1 << j] =
                    min(d[mask | 1 << j], d[mask] + cost[i][j]);
    }
    return d.back();
}
```

实现要点 先估算状态数再开数组。有限棋盘题把坐标关系预处理成合法位转移，不应在每个状态里重新写几何判断。

棋盘状态、子集分配与轮廓 DP 的掩码含义各不相同。位压缩只负责编码；时间仍取决于状态数与转移数，空间还需乘上全部附加维度。

5.7 集合与末尾共同构成的状态

遍历一组地点的路径问题，未来既依赖访问过的集合，也依赖当前停在哪个点。只记录 mask 会合并后继代价不同的状态，必须增加 last。

令 dp[mask][u] 为从指定起点出发，访问集合恰为 mask、最后停在 u 的最小代价。转移到未访问 v，得到 dp[mask|1<<v][v]。若要求回到起点，最后再加回边费用；如果只找路径则直接取末尾最小值。

正确性 每条合法路径都有唯一的最后一步，删去它便得到相应前驱状态。状态包含所有影响未来的信息，枚举全部前驱不会遗漏也不会引入重复访问。

复杂度与接口 O(n²2^n) 时间、O(n2^n) 空间。代码固定从 0 出发，求访问全部点的最短路径，不要求回到 0；n 通常不超过约 20。

```
ll subset_path(const vector<vector<ll>> &w) {
    int n = w.size();
    assert(n > 0 && n < 25);
    int S = 1 << n;
    vector<vector<ll>> dp(S, vector<ll>(n, INF));
    dp[1][0] = 0;
    for (int mask = 1; mask < S; ++mask)
        for (int u = 0; u < n; ++u)
            if (dp[mask][u] != INF)
                for (int v = 0; v < n; ++v)
                    if (!(mask & (1 << v))) {
                        if (w[u][v] == INF)
                            continue;
                        int nxt = mask | (1 << v);
                        dp[nxt][v] = min(dp[nxt][v], dp[mask][u] + w[u][v]);
                    }
    return *min_element(dp.back().begin(), dp.back().end());
}
```

实现要点 断言 n<25 只是防止移位超界，不代表 n=24 的内存一定可接受。不存在的边用 INF；累加必须在 ll 范围内。若允许经过中间地点但暂不算访问，先做最短路闭包时还要解释“经过”与“访问”的题意差别。

5.8 子集枚举与 SOS DP

小集合编码为掩码后，既能枚举其子集，也能把子集统计量批量汇总到超集。可行规模主要由有效位数 B 决定，而非输入对象总数。

枚举 mask 的非空子集，令 sub=(sub−1)&mask。若要对每个 mask 枚举所有子集，总次数是 3^B，因为每一位只有“不在 mask、在 mask 不在 sub、在 sub”三种状态。SOS DP 则逐位决定是否去掉当前位，把 f[mask] 变成所有 sub⊆mask 的总和。

正确性 处理前 i 位后，dp[mask] 累计了只允许在这 i 位上删位的所有子集。下一位同时保留含位状态和不含位状态，两类互不重复。处理全部位即得到完整子集和。

复杂度与接口 子集和 O(B2^B)，空间 O(2^B)。本实现求整数和；求最大值时把加法替换成 max，单位元随之调整。

```
vector<ll> subset_sum_transform(vector<ll> f, int B) {
    assert(int(f.size()) == (1 << B));
    for (int b = 0; b < B; ++b)
        for (int mask = 0; mask < (1 << B); ++mask)
            if (mask & (1 << b))
                f[mask] += f[mask ^ (1 << b)];
    return f;
}
// 包含空子集，最后一次在 sub==0 时结束
void enumerate_subsets(int mask, function<void(int)> visit) {
    for (int sub = mask;; sub = (sub - 1) & mask) {
        visit(sub);
        if (sub == 0)
            break;
    }
}
```

实现要点 B 通常不超过二十多位，实际还取决于每个状态大小与总时限。变换后的数组不再是原始 f；原地逆变换需要逐位减法，模数语义要一致。

5.9 数位 DP 的双边界状态

数位 DP 按高位到低位处理，把上下界限制、前导零和题意自动机状态拆开。只有紧贴边界时下一位范围受限制，脱离边界后才可复用相同状态。

本节 digit_cost 处理等长十进制边界 L、R，在 L≤x≤R 中最小化 x 与两端逐位相等的次数之和。每位代价为两次相等判断，状态仅需位置和两个贴边标记；若加入余数、数字种类或禁串限制，需扩充状态。

复杂度与接口 十进制位数 D，状态 O(D)，转移 O(10D)。

```
int digit_cost(string L, string R) {
    assert(L.size() == R.size());
    int n = L.size();
    vector<array<array<int, 2>, 2>> memo(n);
    for (auto &x : memo)
        for (auto &y : x)
            y.fill(-1);
    function<int(int, int, int)> dfs = [&](int p, int tl, int tr) {
        if (p == n)
            return 0;
        int &ans = memo[p][tl][tr];
        if (ans >= 0)
            return ans;
        ans = 1000000;
```

```
        int lo = tl ? L[p] - '0' : 0, hi = tr ? R[p] - '0' : 9;
        for (int x = lo; x <= hi; x++)
            ans = min(ans, (x == L[p] - '0') + (x == R[p] - '0') +
                           dfs(p + 1, tl && (x == lo), tr && (x == hi)));
        return ans;
    };
    return dfs(0, 1, 1);
}
```

实现要点 不同长度和统计数字本身的性质时需要前导零状态。tight 为真时受当前界约束；下降为假后不能重新变真。

不含某个模式串时可以把 KMP 状态接入数位 DP；整除条件接余数。零是否算合法数字、前导零是否参与模式匹配必须先在状态定义中决定。

5.10 转移设计与优化边界

先定义“状态已经处理了什么”和“答案是否要求选满”。网格 DP 按路径方向处理；树 DP 以父子关系为边界；区间 DP 以长度为顺序；DAG DP 以拓扑为顺序。记忆化只是求值方式，不能让有后效性的状态变成合法 DP。

优化优先删掉冗余维度与重复计算。前缀和加速一段求和，单调队列加速滑动最值，位集合加速布尔可达，滚动数组节约只依赖前几层的空间。斜率优化、分治优化和四边形不等式优化需要各自的单调性条件，不能仅凭递推式看起来相似就使用。

最长公共子序列、编辑距离与路径计数虽然常有相似的二维表，状态含义和合并规则却不同。计算方案数时，应证明每个方案恰好被计入一次，不能直接把最值转移中的 max 替换为加法。

5.11 多维状态的典型模型

编辑距离令 dp[i][j] 表示两个前缀的最少修改次数。删除与插入来自 dp[i−1][j]、dp[i][j−1] 加一，替换来自 dp[i−1][j−1] 加当前字符是否不同；空前缀的代价等于另一边长度。交错串则把数值状态换成布尔可达，并检查从哪一边取下一个字符。去重子序列计数若以字符结束状态为摘要，遇到同字符应替换旧结束状态，避免把同一字符串按不同位置重复计入。

两个资源限制的背包用 dp[a][b]，零一物品的两维都倒序。盈利计划中人数是上限、利润是至少达标，可以将利润维截为 min(P,旧利润+收益)，把所有已经达标的状态合并。骑士留在棋盘上的概率 DP 保存每格概率，向八个方向各分配原概率的 1/8，出界部分丢弃；不能按合法方向数重新归一化，滚动步数维即可。

扰乱字符串需要记两个起点和长度，状态 f[i][j][len] 表示两段能否互相得到。枚举切分长度 k，分别检查不交换两半与交换两半两种组合。长度从小到大计算，朴素 $O(n^4)$ 时间、$O(n^3)$ 空间；字符频次不同可先剪枝。只记一个区间会丢失另一段位置，不能复用最长回文的二维状态。

网格最长递增路径可按数值方向形成 DAG，或用记忆化搜索求每点出发的最长路径，时间 O(nm)。相同值之间没有递增边，因此即使原网格存在几何环，转移图仍无环。树高计数、染色与删除问题则把最后一层或最后一种操作写进状态；每增加一个维度，都应说明它记录了哪项未来必需的信息。

5.12 典型题解析

例题 5.1 炮兵阵地与轮廓状态

洛谷 P2704 基础验证

问题 网格的平原格可以放炮兵，同一行或同一列距离不超过 2 的两门炮会互相攻击，求最大布置数。列数不超过 10。

解法 预生成同行无冲突的 mask，满足 mask&(mask<<1)=0 且 mask&(mask<<2)=0。第 i 行状态 c 还需避开山地，并与前一行 a、前两行 b 不相交。设 dp[i][a][b] 表示最近两行分别是 a、b，枚举 c 转移到 dp[i+1][c][a]。

正确性与复杂度 冲突只跨两行，更早的行不会影响新行，最近两行构成充分状态。每次只增加当前行 popcount，所有合法布置都对应唯一的状态路径。时间 $O(N\,S^3)$、空间 $O(S^2)$，S 为合法行状态数。

示例推演 M=3 时同行状态 101 因水平距离 2 而非法，001、010、100 合法。当前行 001 时，前两行都不能在第一列有炮。

实现要点 本题的两格攻击含距离 1 和 2。只有“上一行状态”的 DP 会漏掉距离 2 的竖向冲突。不可达状态设为负无穷。

练习建议 从只禁止邻行同列的简化版本开始，再增加第二行记忆。

原题与解法依据 洛谷 P2704 基础验证

例题 5.2 Ropeway 用单调队列优化临时修改 DP

2022 ICPC 南京 B

问题 数轴两端是索道站，部分中间位置固定必须建支撑塔，塔间跨度不超过 k。每次询问仅临时改变一个位置的费用，之后恢复，求最低费用。

解法 把站点视为必须选、费用为 0 的位置。f[i] 是最后选中 i 的最小费用，转移取合法前驱窗口中的最小 f[j] 再加 a[i]。

设 last[i] 是 i 之前最后一个必须塔。$f[i]=a[i]+\min f[j]$，其中 $\max(i-k,last[i])\le j<i$。这样既限制跨度，也保证不能跨过未选的必须塔。站点编号为 0 与 n+1，费用为零；到达必须塔后更新 last。

用单调队列维护前驱最小值；遇到必须建塔的点后，后续合法前驱不能越过该点。再从右向左计算后缀对应状态。

预处理 g[i] 为选中 i 之后到终点的最小后缀费，不计 i 的建塔费。只重算 $p \le i \le \min(p+k-1,n+1)$ 的 f[i]，取其中 min(f[i]+g[i])。

必须塔集合始终保持不变；每次只改价，算完恢复 a[p] 及本次改动的 f，不能累计修改。

正确性 任意连续 k 个位置至少选一座塔，否则前后跨度大于 k。因此所有方案都在这段局部窗口选中某个 i。f[i] 与不计 i 费用的 g[i] 分别负责两侧，相加枚举了全部方案；单调队列只删更早过期且不优的候选。

示例推演 费用 [3,1,4,1]、k=2，两端费用 0。选位置 2 和 4，总费 2。若位置 2 临时涨到 10，最优变为选 1、3，总费 7；不能只替换原最优方案中的第二项。

实现要点 必须分清“允许跨过该位置”和“必须在该位置建塔”。查询是临时的，预处理结果不累计改变。局部重算时队列初始前驱窗口须完整。

复杂度 预处理 O(n)，询问局部 O(k)，总体 O(n+qk)，空间 O(n)。

练习建议 先独立实现无修改 O(nk) 与 O(n) 两版并核对，再加入单次改价；最后加入题目完整约束。

原题与解法依据 2022 ICPC 南京 B

例题 5.3 Knapsack 免费物品与付费背包的分界

2023 ICPC 南京 G

问题 n 件物品有体积 w 和价值 v，可先免费选择 k 件获得价值，再从剩余物品中选总体积 $\le W$ 的物品，最大化总价值。

解法 按体积从大到小排序。免费物品和付费物品之间可通过交换，让免费物品位于体积较大的前缀。

枚举前缀分界 i，免费选择前缀价值最大的 k 件，后缀用容量 W 的 0 1 背包。

前缀前 k 大价值和用小根堆维护；背包从后往前加入物品，边扫描边和对应前缀最优值组合。

同体积不必强行认定所有最优方案具有唯一分界，只需证明存在一个能被枚举的最优方案。

正确性 若免费物品 a 的体积小于付费物品 b，互换免费与付费身份不会增大付费体积，总价值不变。前缀中未选择的高价值物品也能替换免费低价值物品。连续交换得到分界形式。

示例推演 物品 (w,v) 为 (5,4),(3,8),(2,6)，k=1、W=3。免费选体积 5 的物品再选体积 3 得 12；免费选体积 3 的物品，再付费选体积 2 得 14。枚举分界不能只试第一个物品。

```
ll free_knapsack(vector<pair<int, ll>> a, int k, int W) {
    int n = a.size();
    assert(0 <= k && k <= n);
    sort(a.begin(), a.end(),
         [](auto x, auto y) { return x.first > y.first; });
    vector<ll> pref(n + 1, -INF);
    priority_queue<ll, vector<ll>, greater<ll>> q;
    ll sum = 0;
    if (!k)
```

```
        pref[0] = 0;
    for (int i = 1; i <= n; i++) {
        q.push(a[i - 1].second);
        sum += a[i - 1].second;
        if ((int)q.size() > k)
            sum -= q.top(), q.pop();
        if ((int)q.size() == k)
            pref[i] = sum;
    }
    vector<ll> dp(W + 1);
    ll ans = pref[n];
    for (int i = n - 1; i >= 0; i--) {
        auto [w, v] = a[i];
        for (int c = W; c >= w; c--)
            dp[c] = max(dp[c], dp[c - w] + v);
        if (pref[i] != -INF)
            ans = max(ans, pref[i] + dp[W]);
    }
    return ans;
}
```

实现要点 0 1 背包容量必须倒序；免费前缀不足 k 件时不能组合。前 k 大是按价值，分界排序是按体积，两个次序不能混淆。

复杂度 使用扫描实现可做到 O(nW+n log k)，空间 O(n+W)。

练习建议 枚举 n≤10 的三个身份“免费、付费、不选”，验证所有分界与重复体积情况。

原题与解法依据 2023 ICPC 南京 G

例题 5.4 Sternhalma 十九格棋盘的状压 DP

2022 CCPC 威海 D

问题 六边形棋盘有 19 个位置，棋子可直接移除，或跳过邻棋并移除它获得被移除位置的分数。多组初始局面求最大得分。

解法 将 19 格编号 0..18，预处理所有合法跳跃三元组 (a,b,c)。mask 记录占用：直接移除 i，收益为零；跳跃要求 a、b 有棋子、c 为空，新状态删除 a、b 并加入 c，收益为 w[b]。

每步棋子数减少一，转移无环。可按 popcount 从少到多递推，也可记忆化搜索；同一棋盘分数下预处理全部状态，询问直接读取 dp[mask]。

正确性 mask 完整描述未来合法动作；各动作都减少 popcount，满足 DP 的无后效性与计算顺序。这里不是因为存在位运算就用状压，而是因为状态空间约 52 万、可穷尽。

示例推演 三个共线格编号 A=0、B=1、C=2，mask=011 表示 A、B 有棋子。跳后为 100，收益 w[B]；直接删 A 则为 010，收益零。

实现要点 六边形坐标不能照搬四方向网格。把几何邻接写在预处理里，并检查反方向跳跃也被枚举。负分跳跃可以不做，因为直接移除收益为 0。

复杂度 $O(2^{19}\times$合法跳跃数$)$，空间 $O(2^{19})$。

练习建议 固定格子编号，手算棋子数不超过三的状态；核对跳跃方向、越界与重复三元组。

原题与解法依据 2022 CCPC 威海 D

第 6 章 连通性最短路与生成树

图的顶点可以是对象，也可以是关系或操作状态。建模时先区分有向与无向、边权类型以及连接是否动态变化，再选择连通性、最短路径或生成树算法。

边权条件	首选方法	使用前先确认
全部为 1	BFS	队列按边数分层，第一次到达即最短
只含 0 和 1	0 1 BFS	0 权边进队首，1 权边进队尾
边权非负	堆优化 Dijkstra	状态扩展后也必须保持非负边权
含负权边	Bellman Ford	需识别从起点可达的负环及其影响

6.1 并查集与离线连通

普通 DSU 维护无向可连接关系，每次合并是把两个等价类合成一个。路径压缩和按大小合并给出近常数的均摊成本；分量摘要如大小、最小值可以随合并同步更新。

复杂度与接口 总复杂度近似 O((n+m)α(n))；离线排序额外 O(m log m)。

```
struct DSU {
    vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b)
            return false;
        if (sz[a] < sz[b])
            swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};
```

实现要点 严格小于 w 与小于等于 w 的询问应使用不同的事件处理顺序。普通 DSU 不支持任意撤销、删边或路径距离。

普通并查集不支持撤销合并。仅有删边且离线已知时，可倒序处理为加边；一般的动态连通性需要回滚并查集与时间分治等方法。有向可达关系不能直接按等价类合并。

岛屿与同行同列石子可先建连通关系。情侣换位可从分量数量推导最少交换；相同值好路径按最大点权排序，再统计同值根；删感染源需区分感染点本身是否也被删除。相同的数据结构服务于不同计数对象，结论必须从题意重新推导。

6.2 带奇偶关系的并查集

关系 DSU 除父亲外，还保存每点到父亲的异或差。路径压缩时累计到根的差值，合并两根时用已知关系反解新根边的差；同根时直接验证关系是否一致。

复杂度与接口 近似 O(α(n)) 每条约束，空间 O(n)。

```
struct ParityDSU {
    vector<int> p, sz, xr;
    explicit ParityDSU(int n) : p(n), sz(n, 1), xr(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        if (p[x] == x)
            return x;
        int old = p[x];
        p[x] = find(old);
        xr[x] ^= xr[old];
        return p[x];
    }
    bool link(int a, int b, int v) {
        int ra = find(a), rb = find(b), d = xr[a] ^ xr[b] ^ v;
        if (ra == rb)
            return d == 0;
        if (sz[ra] < sz[rb])
            swap(ra, rb);
        p[rb] = ra;
        xr[rb] = d;
        sz[ra] += sz[rb];
        return true;
    }
};
```

实现要点 同根时要检查原有关系是否与新约束冲突。它可解决成对相同相反关系，但任意析取条件仍需要 2 SAT。

普通 DSU 合并只关心能否连接，关系 DSU 还必须关心连接后的相对状态。若题目关系为模 k 加法，可把 xor 改成模 k 势能；每一步符号都需要重新推导。

6.3 BFS 多源搜索与洪水填充

无权图每条边代价相同，FIFO 队列按距离层扩展，所以首次到达即最短。多源 BFS 将所有起点距离设为零，等价于增加一个零费用超级源，但不需要实际建点。

复杂度与接口 显式图 O(n+m)，网格 O(nm)。

```
vector<int> bfs(const vector<vector<int>> &g, const vector<int> &sources) {
    vector<int> d(g.size(), -1);
    queue<int> q;
    for (int s : sources)
        if (d[s] < 0)
            d[s] = 0, q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u])
```

```
            if (d[v] < 0)
                d[v] = d[u] + 1, q.push(v);
    }
    return d;
}
```

实现要点 入队时标记，避免同层重复入队。最短路径的全部方案需要记录同层所有前驱，不能第一次到达就丢弃后续前驱。

状态可以是位置加钥匙 mask，也可以是两个角色的位置对。复杂度由展开后的状态数与转移数决定，不能只用原图 n、m 估算。

边界连通区域可先从边界搜索，再处理未访问内部；最远海洋问题可由全部陆地同时出发。删除砖块可倒序恢复并连接屋顶虚拟点。网格搜索深度可能达到格子总数，长链输入应优先采用迭代实现。

6.4 0 1 BFS

边权仅为 0 或 1 时，双端队列把零费用松弛放在前端、一费用放在后端，使队列保持距离近似分层。它处于 BFS 与 Dijkstra 之间，省掉优先队列的对数因子。

复杂度与接口 O(n+m)，空间 O(n)。

```
vector<ll> bfs01(const vector<vector<pair<int, int>>> &g, int s) {
    vector<ll> d(g.size(), INF);
    deque<pair<ll, int>> q;
    d[s] = 0;
    q.push_front({0, s});
    while (!q.empty()) {
        auto [du, u] = q.front();
        q.pop_front();
        if (du != d[u])
            continue;
        for (auto [v, w] : g[u])
            if (d[v] > du + w) {
                assert(w == 0 || w == 1);
                d[v] = du + w;
                if (w)
                    q.push_back({d[v], v});
                else
                    q.push_front({d[v], v});
            }
    }
    return d;
}
```

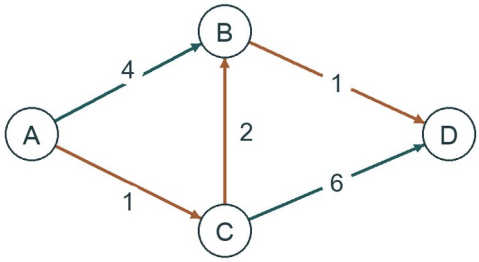
实现要点 边权必须只有 0 或 1。不能在第一次入队时永久锁定节点，因为之后可能得到更短距离。

方向改变次数、免费与收费通道都可能形成 0 1 边。边权为任意正整数时双端队列的排序性质消失，应改用 Dijkstra。

二维接雨水需按当前水位的小根堆从边界扩展，每个新格水位取外侧水位与格高的较大者。它是瓶颈最短路式搜索，不能使用只适合 0 1 边权的双端队列规则。

6.5 Dijkstra 与状态扩展

Dijkstra 每次确定暂定距离最小的顶点。由于边权非负，经尚未确定的顶点绕行不会缩短它的距离。堆中允许保留过时条目，弹出时用当前 dist 检查并跳过。



确定点	B	C	D
A	4	1	∞
C	3	1	7
B	3	1	4
D	3	1	4

从 A 出发，依次确定 A、C、B、D；最终到 D 的最短距离为 4。

图 6.1 最短路径的松弛与顶点确定

复杂度与接口 O((n+m) log n)，空间 O(n+m)；扩展图按状态数重算。

```
vector<ll> dijkstra(const vector<vector<pair<int, ll>>> &g, int s) {
    vector<ll> d(g.size(), INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                   greater<pair<ll, int>>>
        q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty()) {
        auto [du, u] = q.top();
        q.pop();
        if (du != d[u])
            continue;
        for (auto [v, w] : g[u]) {
            assert(w >= 0);
            if (w >= INF || du > INF - w)
                continue;
            if (d[v] > du + w)
                d[v] = du + w, q.push({d[v], v});
        }
    }
    return d;
}
```

实现要点 有负边不能套。本实现 INF=2^{62}，要求真实可达距离小于 INF，且边权非负。A 星还需要启发函数不会高估；四邻格子用曼哈顿距离且每步费用至少为 1 时可用。

分层图将已用优惠次数、交通方式等限制写入状态，再在扩展图上求最短路。A 星还依赖不高估剩余代价的启发函数，并须配合正确的更新与终止规则。

最小体力路径把路径代价的加法改为 max(当前瓶颈,新边权)，仍具有扩展后代价不减的性质。钥匙迷宫可用 (格子,mask) 做 BFS；电动车可用 (城市,电量) 做 Dijkstra，分别为充电和行驶建边。

6.6 Bellman Ford 与 Floyd

Bellman Ford 按边松弛，限定最多 k 条边时每轮必须读取上一轮副本；否则同一轮可能串联多条边。Floyd 按允许的中间点集合逐步扩展，循环顺序 k 在最外层。

复杂度与接口 Bellman Ford O(nm)，Floyd $O(n^3)$ 与 $O(n^2)$ 空间。

```
struct Edge {
    int u, v;
    ll w;
};
pair<vector<ll>, bool> bellman(int n, vector<Edge> es, int s) {
    vector<ll> d(n, INF);
    d[s] = 0;
    bool changed = false;
    for (int k = 0; k < n; k++) {
        changed = false;
        for (auto e : es)
            if (d[e.u] != INF) {
                i128 nd = (i128)d[e.u] + e.w;
                assert(-INF < nd && nd < INF);
                if (nd < d[e.v])
                    d[e.v] = (ll)nd, changed = true;
            }
        if (!changed)
            break;
    }
    return {d, changed}; // true 表示起点可达负环。
}
void floyd(vector<vector<ll>> &d) {
    int n = d.size();
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] != INF && d[k][j] != INF) {
                    i128 nd = (i128)d[i][k] + d[k][j];
                    assert(-INF < nd && nd < INF);
                    d[i][j] = min(d[i][j], (ll)nd);
                }
}
```

实现要点 SPFA 最坏复杂度仍为 O(nm)，不能把它当作稳定线性算法。有负环时最短路可能不存在；要说明负环是否必须从起点可达。

SPFA 是队列化松弛策略，不具有一般线性最坏界。若从单源检测可达负环，可做 n 轮松弛；检测全图任意负环可引入超级源。不可达距离与 INF 相加前必须保护。

6.7 Prim 与 Kruskal 最小生成树

Kruskal 按边权从小到大加入不成环的边。当前各连通分量形成若干棵树，加入连接不同分量的最小边，存在某个最优生成树包含这一步选择。

Prim 按割性质选择连接当前树与外部的最小边。稀疏图可用堆维护，稠密图可扫描最小跨边，时间为 $O(n^2)$。Prim 比较接入树的单条边权，Dijkstra 比较从源点累计的路径长度。

复杂度与接口 Kruskal O(m log m)，依赖 DSU 与 Edge；堆 Prim O(n+m log(m+1))。Prim 输入 0 下标无向邻接表，每边双向加入；返回 i128 总权，不连通时为空 optional。

```
optional<ll> kruskal(int n, vector<Edge> es) {
    sort(es.begin(), es.end(), [](auto a, auto b) { return a.w < b.w; });
    DSU d(n);
    ll ans = 0;
    int used = 0;
    for (auto e : es)
        if (d.unite(e.u, e.v))
            ans += e.w, ++used;
    if (used != n - 1)
        return nullopt;
    return ans;
}
optional<i128> prim(const vector<vector<pair<int, ll>>> &g) {
    int n = g.size();
    if (!n)
        return i128(0);
    vector<int> used(n);
    int count = 0;
    i128 cost = 0;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                   greater<pair<ll, int>>>
        q;
    q.push({0, 0});
    while (!q.empty()) {
        auto [w, u] = q.top();
        q.pop();
        if (used[u])
            continue;
        used[u] = 1;
        ++count;
        cost += w;
        for (auto [v, weight] : g[u])
            if (!used[v])
                q.push({weight, v});
    }
    if (count != n)
        return nullopt;
    return cost;
}
```

实现要点 不连通时返回无解。最小生成树也是最小瓶颈生成树，反向不成立；若只求瓶颈可以只保留最后一条选中边权。

最小生成树最小化全部选边总和，最短路树最小化源到各点的距离，两者不同。离线阈值连通查询可以同样按边权扫描，却不需要真的选出生成树边。

供水题给每个村庄与虚拟节点连接打井费用，把建设选择统一成 MST。邻接表与链式前向星都是建图方式；重边、自环、无向边双向插入和数组容量属于输入建模，不能忽略。

6.8 离线阈值与状态图建模

边权阈值连通可将边和询问按阈值排序，逐步加入边并回答 DSU 查询。秘密传播一类时间分组必须处理同一时间的全部连通关系，再清除不与已知源连接的临时信息，不能让未传播成功的关系永久保留。

状态图将题目限制编码在节点中：钥匙集合、剩余优惠次数、角色位置与已使用资源都是常见维度。先证明这些状态足以决定下一步，再选择 BFS、0 1 BFS 或 Dijkstra。双向 BFS 适合可逆且能正确列出反向前驱的搜索；A 星还要求启发不高估剩余代价。

6.9 典型题解析

例题 6.1 Gifts from Knowledge 相同相反约束

2023 ICPC 济南 G

问题 每行可以左右翻转，要求 01 矩阵每一列最多一个 1，求行翻转方案数。

解法 对列 j 和镜像列 m−j+1 成对统计 1。若总数超过 2，则必然无法放入两列；奇数中间列单独限制≤1。

恰有两个 1，且来自不同行时：原来同列要求两行翻转状态相反，原来不同列要求状态相同。

两个 1 来自同一行而位于镜像两列时，无需加状态约束；它们无论翻不翻都仍各占一列。

将约束输入奇偶 DSU。矛盾则 0；否则每个独立行状态连通块可任选一个基准状态，答案是 2^块数。

正确性 一对镜像列内的 1 只会交换左右位置，要求不同最终列恰是一个二元 XOR 关系。不同镜像对之间共享的唯一变量是各行翻转状态，因此全部约束可以合并到同一张关系图。

示例推演 两行的 1 都在左侧镜像列，则 state[1] xor state[2]=1；第三行与第二行的 1 原在相反列，则 state[2] xor state[3]=0。这个块有 2 种整体状态，不是 4 种。

实现要点 不要每个镜像对都把方案数乘 2，变量之间可能连接或产生奇环矛盾。孤立行仍各有两种翻转状态。中间列不能按双列容量处理。

复杂度 $O(nm+n\alpha(n))$，空间 $O(n+m)$。

练习建议 先用 BFS 二染色解决纯相同相反图，再写 ParityDSU；$n\le 8$ 枚举所有 2^n 行翻转并核对。

原题与解法依据 2023 ICPC 济南 G

例题 6.2 Bitwise And Path 对答案掩码建连通性

2025 ICPC 南京 F

问题 动态给无向图加边，询问两点之间路径边权 AND 的最大值；边权只有 12 个有效二进制位。

解法 mask 可行当且仅当只保留包含其全部置一位的边后，两点连通。为 2^12 个 mask 各建 DSU，新边加入边权 w 的全部子掩码。询问自高位向低位尝试加一位，连通就保留。

递归更新时，某 mask 下两点已经连通即可停止向更小子掩码传播；此前的成功合并已完成传播。查询先检查零掩码连通，无法到达时按题面约定输出。

正确性 掩码包含关系把“路径整体 AND”换成“边是否全部符合条件”。一个较高二进制位的权值大于全部低位总和，因此高位优先的答案贪心正确。更新剪枝的正确性依赖每一次成功合并都曾继续传播给所有子掩码。

示例推演 边权 1101 和 1110 形成的两边路径 AND=1100。查询试 1000、1100 均连通，试 1110 不连通，故最高结果保留 1100。

实现要点 只更新边权 w 对应的 DSU 会漏子掩码；B=12 是可行性的关键，不能直接扩到 60 位。

复杂度 摊还约 $O(\alpha(n)(n2^B B+qB))$，空间 $O(n2^B)$，$B=12$。

练习建议 先在 $B\le 5$ 时暴力维护全部子图，核对加边与查询，再加入递归剪枝。

原题与解法依据 2025 ICPC 南京 F

第 7 章 拓扑缩点匹配与网络流

有向图可通过强连通分量消去环，拓扑序确定处理顺序；无向图的桥与割点揭示连通瓶颈。匹配、网络流和 2 SAT 分别表达一对一选择、容量限制和二元逻辑约束。

7.1 拓扑排序与 DAG DP

DAG 中每条边规定一种先后关系。拓扑序让所有前驱先处理，适合最长路、方案数和依赖完成时间。Kahn 算法每次取入度零节点，最后处理数量小于 n 表明存在环。

复杂度与接口 O(n+m)，最小字典序为 O((n+m) log n)。

```
vector<int> topo(const vector<vector<int>> &g) {
    int n = g.size();
    vector<int> deg(n), ord;
    queue<int> q;
    for (auto &vs : g)
        for (int v : vs)
            ++deg[v];
    for (int i = 0; i < n; i++)
        if (!deg[i])
            q.push(i);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ord.push_back(u);
        for (int v : g[u])
            if (--deg[v] == 0)
                q.push(v);
    }
    if ((int)ord.size() != n)
        return {};
    return ord;
}
```

实现要点 出边方向必须是前置任务到后续任务。DAG 路径计数先定起点、终点与是否包含零边路径。功能图的剥叶和环处理与普通 DAG 不完全相同。

队列中同时存在多个可选节点是拓扑序不唯一的信号。若需要字典序最小拓扑序，把队列换成小根堆；计数不同拓扑序不能靠一个队列直接完成。

食物链计数可用 DAG 方案数，反向盖章可以维护尚未满足的字符依赖。每点出度为一的功能图，可先剥去非环节点，再逐个处理环，树枝信息按删除顺序汇总。最大邀请人数还需比较最长环与二环所接树枝；周期交通状态可按余数与已访问状态判断循环。

7.2 强连通分量与缩点 DAG

有向图中的“相互可达”是一种等价关系。把每个最大相互可达集合压成一个点，原图的所有环都被吸收，剩下的图一定无环。随后可以用拓扑序做路径 DP。

本节采用 Kosaraju 算法。第一遍在原图记录 DFS 完成顺序，第二遍按其逆序搜索反图，每次搜索得到一个强连通分量。显式栈可避免长链图耗尽递归栈。

正确性 若缩点图中仍有环，环上各分量将相互可达，与“不同强连通分量”矛盾。第二遍的起点来自尚未处理分量中适当的拓扑位置，反图搜索不会越过未处理分量的边界，却能覆盖同一分量。

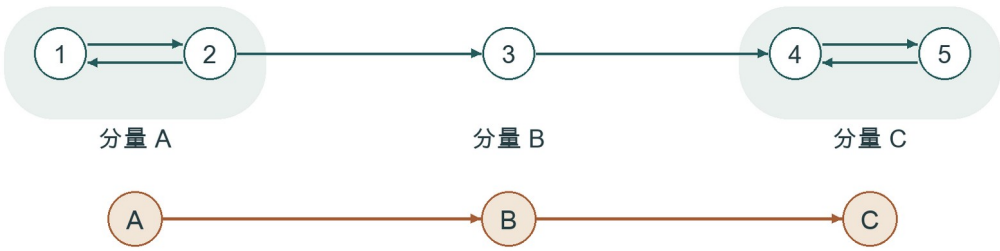


图 7.1 强连通分量压缩为有向无环图

复杂度与接口 两遍搜索和缩点均为 O(n+m)，空间 O(n+m)。节点 0..n−1；返回每点所属分量，分量号不作为拓扑顺序直接使用。

```
vector<int> scc(const vector<vector<int>> &g) {
    int n = g.size();
    vector<vector<int>> rg(n);
    for (int u = 0; u < n; ++u)
        for (int v : g[u])
            rg[v].push_back(u);
    vector<int> seen(n), order;
    for (int s = 0; s < n; ++s)
        if (!seen[s]) {
            vector<pair<int, int>> st{{s, 0}};
            seen[s] = 1;
            while (!st.empty()) {
                int u = st.back().first;
                int &i = st.back().second;
                if (i == int(g[u].size())) {
                    order.push_back(u);
                    st.pop_back();
                } else {
                    int v = g[u][i++];
                    if (!seen[v]) {
                        seen[v] = 1;
                        st.push_back({v, 0});
                    }
                }
            }
        }
    vector<int> comp(n, -1);
    int id = 0;
    reverse(order.begin(), order.end());
    for (int s : order)
        if (comp[s] == -1) {
            vector<int> st{s};
            comp[s] = id;
            while (!st.empty()) {
```

```
                int u = st.back();
                st.pop_back();
                for (int v : rg[u])
                    if (comp[v] == -1) {
                        comp[v] = id;
                        st.push_back(v);
                    }
            }
            ++id;
        }
    return comp;
}
```

实现要点 SCC 适用于有向图；无向连通块只需 DFS 或 DSU。点权全部非负时，可以把分量权求和；有负权、不能重复经过或存在资源限制时，压成“总点权”需要重新证明。缩点边去重可减小常数，也可保留重边并准确计算入度。

7.3 桥与割点的 lowlink 判定

无向图中，删除一条边让连通分量数增加，该边是桥；删除一个点让连通分量数增加，该点是割点。两者都利用 DFS 子树是否能通过返祖边绕回上方。

dfn[u] 是访问时间，low[u] 是从 u 的 DFS 子树经树边向下、再至多用一条返祖边能到达的最小时间。树边 $u\to v$ 是桥当且仅当 low[v]>dfn[u]。非根 u 是割点当某个孩子满足 low[v]≥dfn[u]；DFS 根则必须有至少两个孩子。

正确性 严格大于表示子树连 u 自身都无法绕回，父子边不可替代；大于等于表示去掉 u 后，该子树无法到达 u 的祖先。根没有祖先，必须单独根据独立搜索分支数量判断。

复杂度与接口 O(n+m) 时间和空间。图的边有唯一编号，同一无向边的两个方向共用编号；允许平行边。

```
struct Lowlink {
    vector<vector<pair<int, int>>> g;
    vector<int> dfn, low, cut, bridges;
    int timer = 0;
    Lowlink(int n) : g(n), dfn(n), low(n), cut(n) {}
    void edge(int u, int v, int id) {
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }
    void dfs(int u, int parent_edge = -1) {
        dfn[u] = low[u] = ++timer;
        int children = 0;
        for (auto [v, id] : g[u]) {
            if (id == parent_edge)
                continue;
            if (!dfn[v]) {
                ++children;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u])
                    bridges.push_back(id);
                if (parent_edge != -1 && low[v] >= dfn[u])
                    cut[u] = 1;
            } else
```

```
                low[u] = min(low[u], dfn[v]);
        }
        if (parent_edge == -1 && children > 1)
            cut[u] = 1;
    }
    void run() {
        for (int u = 0; u < int(g.size()); ++u)
            if (!dfn[u])
                dfs(u);
    }
};
```

实现要点 不能只跳过父节点：平行边中另一条边仍是合法返祖边。此紧凑版使用递归，长链应换成显式栈或确保运行环境的栈容量。删除所有桥后做连通块，可得到边双连通分量及桥树。

7.4 Hopcroft Karp 二分图匹配

二分图匹配每次沿未匹配边从左到右、已匹配边从右到左形成增广路，翻转后匹配数加一。Hopcroft Karp 用 BFS 分层后同时找多条最短增广路。



图 7.2 沿增广路翻转匹配状态

复杂度与接口 O(m√n)，空间 O(n+m)。

```
struct HopcroftKarp {
    int n, m, shortest;
    vector<vector<int>> g;
    vector<int> L, R, d;
    HopcroftKarp(int n, int m) : n(n), m(m), g(n), L(n, -1), R(m, -1), d(n) {}
    bool bfs() {
        queue<int> q;
        fill(d.begin(), d.end(), INT_MAX);
        shortest = INT_MAX;
        for (int u = 0; u < n; u++)
            if (L[u] < 0)
                d[u] = 0, q.push(u);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (d[u] >= shortest)
                continue;
            for (int v : g[u]) {
```

```
                if (R[v] < 0)
                    shortest = d[u] + 1;
                else if (d[R[v]] == INT_MAX)
                    d[R[v]] = d[u] + 1, q.push(R[v]);
            }
        }
        return shortest != INT_MAX;
    }
    bool dfs(int u) {
        for (int v : g[u]) {
            if ((R[v] < 0 && d[u] + 1 == shortest) ||
                (R[v] >= 0 && d[R[v]] == d[u] + 1 && dfs(R[v]))) {
                L[u] = v;
                R[v] = u;
                return true;
            }
        }
        d[u] = INT_MAX;
        return false;
    }
    int solve() {
        int ans = 0;
        for (int x : L)
            ans += x >= 0;
        while (bfs())
            for (int u = 0; u < n; u++)
                if (L[u] < 0 && dfs(u))
                    ++ans;
        return ans;
    }
};
```

实现要点 左右编号是独立的 0 开始。递归 DFS 在极长交替路中要注意栈空间；这与普通并查集或相同相反约束不同。

匹配只解决一对一选择约束。一般图没有左右部时不能直接使用；一项可匹配多次可改成容量或复制节点。残余图的可达性还能判断可替换选择与最小点覆盖。

7.5 最大流与最小割建模

最大流描述带容量的资源传递：除源汇外每个节点流入等于流出。残余图保存继续增加流量以及撤销旧分配的可能。最小割则把节点分成源侧与汇侧，割掉的容量总和限制任何可行流。

Dinic 先在残余图中 BFS 分层，再沿层数递增的边发送阻塞流；本轮无法继续增广后重新分层。当前弧记录已处理的边，避免反复扫描。每次送流同时更新正向与反向残余容量。

正确性 每次增广保持容量限制和流量守恒。终止时残余图中源无法到汇，源可达集合到外部的正向边全部饱和，反向跨割流为零，因此当前流等于这个割的容量，达到最优。

复杂度与接口 一般图最坏 $O(V^2E)$，空间 $O(V+E)$；二分图单位容量通常更快。接口拒绝自环，平行边可直接加入；容量和总流量应在 Π 范围内。

```
struct Dinic {
    struct E {
        int to, rev;
        ll cap;
    };
    vector<vector<E>> g;
    vector<int> level, ptr;
    Dinic(int n) : g(n), level(n), ptr(n) {}
    void add(int u, int v, ll c) {
        assert(u != v && c >= 0);
        E a{v, int(g[v].size()), c};
        E b{u, int(g[u].size()), 0};
        g[u].push_back(a);
        g[v].push_back(b);
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        q.push(s);
        level[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto e : g[u])
                if (e.cap && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
        }
        return level[t] != -1;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t)
            return f;
        for (int &i = ptr[u]; i < int(g[u].size()); ++i) {
            E &e = g[u][i];
            if (!e.cap || level[e.to] != level[u] + 1)
                continue;
            ll z = dfs(e.to, t, min(f, e.cap));
            if (!z)
                continue;
            e.cap -= z;
            g[e.to][e.rev].cap += z;
            return z;
        }
        return 0;
    }
    ll flow(int s, int t) {
        assert(s != t);
        ll ans = 0;
        while (bfs(s, t)) {
            fill(ptr.begin(), ptr.end(), 0);
            while (ll z = dfs(s, t, INF))
                ans += z;
        }
        return ans;
    }
};
```

实现要点 边权是费用时不能直接当容量；最小费用流是另一种模型。所谓无限容量应超过所有有限割的可能总和，又不能导致加法溢出。这里只给最大流，不把它作为所有铜牌题的必备前置。

7.6 二元逻辑约束与 2 SAT

奇偶 DSU 处理相同或相反关系；当条件变成“两个命题至少一个为真”，可以用 2 SAT。每个布尔变量拆成真、假两个文字，文字与其否定的编号相差一位。

子句 a 或 b 等价于 ¬a→b 与 ¬b→a。建立蕴含图，若某变量的真与假处于同一个 SCC，就会互相强制，无法满足。否则按缩点拓扑顺序给相反文字赋值，可以构造一组解。

正确性 矛盾时真能推出假、假也能推出真。无矛盾时在缩点 DAG 中从后向前选择尚未赋值文字为真、其否定为假，可避免已选真命题指向假命题。本节按 Kosaraju 的分量顺序等价地比较编号。

复杂度与接口 O(n+m) 时间与空间。依赖前面的 scc；它给出的分量编号从拓扑源向汇增加，此约定不可换成另一份 SCC 后仍沿用。

```
struct TwoSAT {
    int n;
    vector<vector<int>> g;
    TwoSAT(int n) : n(n), g(2 * n) {}
    void either(int i, int a, int j, int b) {
        int x = 2 * i + a, y = 2 * j + b;
        g[x ^ 1].push_back(y);
        g[y ^ 1].push_back(x);
    }
    vector<int> solve() {
        auto c = scc(g);
        vector<int> ans(n);
        for (int i = 0; i < n; ++i) {
            if (c[2 * i] == c[2 * i + 1])
                return {};
            ans[i] = c[2 * i + 1] > c[2 * i];
        }
        return ans;
    }
};
```

实现要点 a、b 必须为 0 或 1，变量从 0 开始。返回空数组表示无解，要求 n>0。一个子句有三个任意文字时已不是 2 SAT；不要用不完整拆边冒充原约束。作为伸展知识，在基本关系图稳定后学习。

7.7 图模型的适用边界

一对一选择使用匹配，可拆分且带数量上限的资源分配使用流，两个布尔文字组成的或约束使用 2 SAT。一般图匹配与最小费用流需要更专门的算法，不能直接套用本章实现。

缩点、桥树与拓扑常作为后续 DP 的预处理。压缩时必须定义一个分量如何与外部交互：能否任意选入口出口、能否收集全部点权、是否保持距离。只有证明这些语义后，“压成一个点”才是正确算法。

7.8 典型题解析

例题 7.1 缩点后的最大点权路径

洛谷 P3387 基础验证

问题 有向图点权非负，允许重复经过点或边，但每个点权仅计算一次，求一条行走路线能取得的最大权和。

解法 求 SCC，并把每个分量内点权相加。强连通分量内可在离开前走到所有点；缩点后按拓扑序做 dp[v]=max(dp[v],dp[u]+weight[v])。起点任意，因此每个分量先用自身点权初始化。

正确性与复杂度 同分量内部相互可达且允许重复行走，可以先收集全部非负点权再前往出口。不同分量在 DAG 中不能回头，路径 DP 正好枚举全部分量序列。总时间 O(n+m)。

示例推演 $1\to2\to1$ 的点权为 3、5，再由 $2\to3$、3 的权为 4。缩点后是权 8 的点指向权 4 的点，答案 12。

实现要点 若点权可以为负，直接收集整个分量不一定最优；如果不允许重复经过，同分量内也未必能走遍。这两条题意是缩点求和的关键。

练习建议 改变这两项条件，给出直接缩点求和失效的反例。

原题与解法依据 洛谷 P3387 基础验证

例题 7.2 网络最大流与容量瓶颈

洛谷 P3376 基础验证

问题 有向网络给出源点、汇点和每条边的容量，求最多能从源传到汇的总量。

解法 按原容量建立正向边，反向边初始为零。用 Dinic 建层并送阻塞流。求完后在残余图从源 BFS，可达侧与不可达侧构成一个最小割，可作为实现结果的独立解释。

正确性与复杂度 任何流都不能超过任意割的容量；残余图已无源汇路径时，当前流恰等于源可达集合形成的割容量，因此最优。一般 Dinic 的界为 $O(V^2E)$。

示例推演 $s\to a$ 容量 3、$s\to b$ 容量 2，两点到 t 的容量均为 4。源点出边总容量是 5，能实际送到 5，最小割也为 5。

实现要点 流量可通过反向边撤销，不能把旧路径视为固定不可改变。并行边不冲突，零容量边可忽略，自环对源汇流没有作用。

练习建议 小图枚举全部割，核对最大流等于最小割。

原题与解法依据 洛谷 P3376 基础验证

例题 7.3 割点与根结点的特殊性

洛谷 P3388 基础验证

问题 无向图可能不连通，求删除后会增加连通分量数的全部节点。

解法 对每个未访问点开启 DFS。非根 u 若有孩子 v 的 low[v]≥dfn[u]，则 u 是割点；根只在 DFS 孩子数至少两个时为割点。输出标记而非每次满足条件就输出，以免重复。

正确性与复杂度 某子树无法越过 u 到达祖先时，删 u 使其与上方分离。根没有上方部分，只有多个彼此独立的 DFS 分支才会分裂。总时间 O(n+m)。

示例推演 链 1−2−3 的割点是 2。三角形 1−2−3−1 没有割点，即使 DFS 根在无向图中的度数为 2，它也只有一个 DFS 孩子。

实现要点 根的无向度数不等于 DFS 孩子数。图不连通时只从 0 搜一次会漏掉其余分量。平行边用边编号处理。

练习建议 小图依次删除每个点重算连通分量数，与 lowlink 标记核对。

原题与解法依据 洛谷 P3388 基础验证

例题 7.4 两个选项组成的逻辑选择

洛谷 P4782 基础验证

问题 若干布尔变量，每条条件要求两个指定真假命题中至少一个成立，判断可满足性并构造方案。

解法 将每条条件拆成两条蕴含边。求 SCC 检查变量真假是否同分量；否则按已说明的分量顺序给值。输出后可再次遍历全部子句检查答案，这一步是低成本的构造验证。

正确性与复杂度 蕴含图完整表达每个二元子句。真假同 SCC 构成双向强制矛盾；无矛盾时缩点 DAG 的逆序赋值存在满足解。复杂度 O(n+m)。

示例推演 条件 (x=1 或 y=1) 与 (x=0 或 y=1) 一起强制 y=1；x 可任选。若再加入 y=0 的单位子句，就无解。

实现要点 单位子句可以把同一文字写两次。SCC 编号方向与实现有关，换模板后必须重新检查构造取值规则。

练习建议 n≤12 枚举所有真假赋值，比对无解判断并验证返回方案。

原题与解法依据 洛谷 P4782 基础验证

例题 7.5 Equivalent Rewriting 只保留最后赋值

2023 ICPC 南京 F

问题 操作 i 把某些变量赋值为 i，初始顺序 1..n。求另一执行顺序，保证所有变量最终值相同。

解法 对每个变量记录原顺序最后一次修改它的操作 last。之前改过该变量的操作都必须在 last 之前。

这样得到的有向边从小编号指向大编号，原序列是一个拓扑序。问题变为能否找到不同的拓扑序。

只需检查是否存在某对相邻操作 i、i+1 之间没有依赖边。有则交换这两项，其他相对次序不变。

不用显式建立全部图：只检查操作 i 修改的变量里是否有 last 等于 i+1。

正确性 交换相邻两项只改变它们的相对先后；没有从 i 到 i+1 的边时所有依赖仍满足。如果每对相邻项都有边，就形成包含所有点的链，拓扑序只能唯一。

示例推演 操作 1 改 x，操作 2 改 y，操作 3 改 x。约束只有 $1 \to 3$，因此交换 1、2 后 [2,1,3] 仍有 x=3、y=2。

实现要点 不用将每个变量的所有修改两两连边，那会产生不必要的平方开销。未被最后操作覆盖的赋值影响必须由 last 精确保留。

复杂度 O(n+修改变量总数)，空间 O(变量总数)。

练习建议 $n \le 8$ 枚举操作排列核对；再尝试给每次操作赋一般不同值，观察依赖条件为何仍成立或何时改变。

原题与解法依据 2023 ICPC 南京 F

例题 7.6 I Just Want One More 匹配增量与残余可达

2023 ICPC 济南 E

问题 在平衡二分图中恰好加一条左右两侧之间的边，使最大匹配数增加，求可以添加的边数。

解法 先求最大匹配，完美匹配时答案为零。交替残余图中，未匹配边从左到右、匹配边从右到左；从全部未匹配左点出发，统计可达左点数 cntL。

从全部未匹配右点沿残余图的反图搜索，统计能到达它们的右点数 cntR。连接前一种左点与后一种右点，即可形成新的增广路，答案为 $\text{cntL} \times \text{cntR}$。

正确性 新增一条边最多增加一个匹配，成功增广必要求其两端分别处于上述两类。反过来，两段旧残余路径若相交，原图就已存在增广路；因此它们不交，能通过新边连接成合法增广路。两端之间也不可能已有边，否则同样与原匹配最大矛盾。乘积恰好统计全部合法新增边。

示例推演 左右各两点，只有 L1−R1。原匹配数为 1，新增 L2−R2 能增大；单独新增 L1−R2 或 L2−R1 只能换配对，匹配数仍为 1。

实现要点 已求出的匹配必须最大，否则残余图的源汇本来就可能连通。正向可达与反向可达方向不能写反。统计的是左右两类点，不包含源汇。

复杂度 $O(m\sqrt{n}+n+m)$，空间 O(n+m)。

练习建议 先做小规模二分图匹配基础题，再枚举所有未存在边重新求匹配，作为 $n \le 8$ 的对拍真值。

原题与解法依据 2023 ICPC 济南 E

第 8 章 树上动态规划与路径维护

根、父子关系与 DFS 序构成树上算法的基本结构。子树可映射为连续区间，路径可拆成重链区间，换根 DP 则通过跨边转移联系所有根的答案。

8.1 迭代树序与不相邻点最大和

树的父子顺序先由遍历建立，再逆序处理孩子。最大权独立集有选与不选两种状态：选 u 时孩子不能选；不选 u 时每个孩子可自由取最优。

复杂度与接口 O(n)，空间 O(n)。

```
ll tree_independent(const vector<vector<int>> &g, const vector<ll> &w) {
    int n = g.size();
    if (!n)
        return 0;
    vector<int> p(n, -1), ord{0};
    p[0] = 0;
    for (int i = 0; i < n; i++) {
        int u = ord[i];
        for (int v : g[u])
            if (v != p[u])
                p[v] = u, ord.push_back(v);
    }
    vector<ll> yes = w, no(n);
    for (int i = n - 1; i > 0; i--) {
        int u = ord[i], f = p[u];
        yes[f] += no[u];
        no[f] += max(yes[u], no[u]);
    }
    return max(yes[0], no[0]);
}
```

实现要点 接口要求输入是连通无向树。选不选状态处理的是独立集，不是所有树 DP 的统一状态；有负权时不选所有点也合法。

树上的状态需要明确“子树内部”和“与父亲相连的接口”。一般树与二叉树的差别主要是孩子数，不能在每个点只处理左右两项。

非负权树的直径可由两次最远点搜索求得，无权树可用 BFS。二叉树平衡性通过高度自底向上判断；完全性可由层序中首次缺少孩子的位置判定，完整子树的高度结构还可用于加速节点计数。

8.2 倍增 LCA 与树上距离

倍增 up[k][u] 保存 u 的 2^k 级祖先，先把深度较大者提到相同深度，再从高位到低位同时跳过仍不同的祖先。最后的父亲就是 LCA。

复杂度与接口 预处理 O(n log n)，LCA O(log n)，空间 O(n log n)。

```
struct LCA {
    int n, K;
    vector<int> dep;
    vector<vector<int>> up;
    LCA(const vector<vector<int>> &g, int root = 0)
        : n(g.size()), K(1), dep(n) {
        while ((1LL << K) <= n)
            ++K;
        up.assign(K, vector<int>(n, root));
        vector<int> p(n, -1), ord{root};
        p[root] = root;
        for (int i = 0; i < n; i++) {
            int u = ord[i];
            up[0][u] = p[u];
            for (int v : g[u])
                if (v != p[u])
                    p[v] = u, dep[v] = dep[u] + 1, ord.push_back(v);
        }
        for (int k = 1; k < K; k++)
            for (int v = 0; v < n; v++)
                up[k][v] = up[k - 1][up[k - 1][v]];
    }
    int get(int u, int v) const {
        if (dep[u] < dep[v])
            swap(u, v);
        int d = dep[u] - dep[v];
        for (int k = 0; k < K; k++)
            if (d >> k & 1)
                u = up[k][u];
        if (u == v)
            return u;
        for (int k = K - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
                u = up[k][u], v = up[k][v];
        return up[0][u];
    }
    int dist(int u, int v) const {
        return dep[u] + dep[v] - 2 * dep[get(u, v)];
    }
};
```

实现要点 输入为从 0 编号的非空连通无权树；根祖先设为自身。加权树用根距离相加再减去两倍 LCA 根距离。

树上距离为 depth[u]+depth[v]−2depth[lca]，加权树用根距离代替深度。二叉搜索树的比较大小 LCA 解法依赖搜索树性质，不能用于一般树。

进入节点时记录 tin，完成全部后代后记录 tout，则子树对应 [tin,tout)。按 tin 存放点值后，可用树状数组维护子树和。该连续性依赖 DFS 的进入、退出次序，普通 BFS 序不具有这一性质。

8.3 DFS 序与重链剖分

DFS 序将子树映射为单一区间；重儿子优先的 DFS 序还使每条重链连续，从而把任意路径拆成 O(log n) 个区间，再接入区间数据结构。

第一遍计算父亲、深度、子树大小以及最大的孩子 heavy。第二遍优先走 heavy，并继承链顶 top；轻儿子另开一条链。路径查询每次处理链顶较深的一侧，然后跳到该链顶的父亲，直到两点同链。

正确性 沿轻边向上时，进入的父子树规模至少翻倍：轻儿子的大小不超过父子树的一半。任何根路径至多有 O(log n) 条轻边，所以拆出的区间数也是 O(log n)。子树在重儿子优先的 DFS 中依然连续。

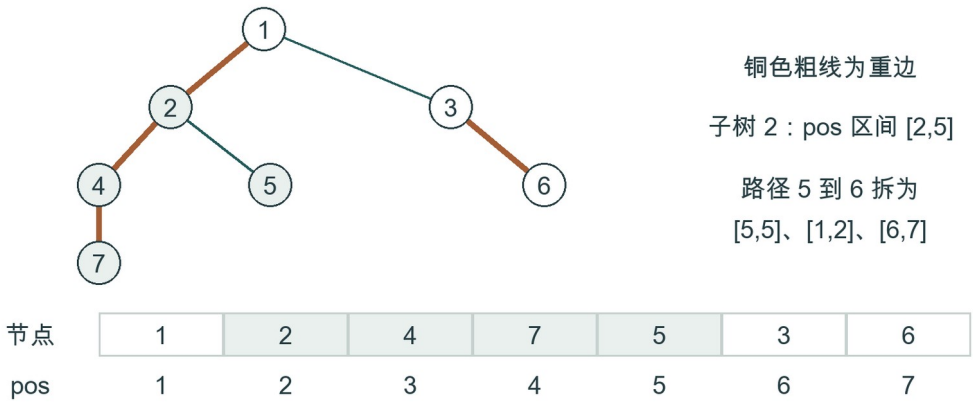


图 8.1 重链连续性与子树区间

复杂度与接口 预处理 O(n)，路径拆分 O(log n)，配合线段树为 O(log² n)；子树查询只需 O(log n)。代码采用迭代遍历，g 为 0 下标无向树。

```
struct HLD {
    int n, timer = 0;
    vector<int> par, dep, sz, heavy, top, pos;
    HLD(const vector<vector<int>> &g, int root = 0)
        : n(g.size()), par(n, -1), dep(n), sz(n, 1), heavy(n, -1), top(n),
          pos(n) {
        vector<int> order{root};
        par[root] = root;
        for (int i = 0; i < n; ++i) {
            int u = order[i];
            for (int v : g[u])
                if (v != par[u]) {
                    par[v] = u;
                    dep[v] = dep[u] + 1;
                    order.push_back(v);
                }
        }
        for (int i = n - 1; i > 0; --i) {
            int u = order[i], p = par[u];
            sz[p] += sz[u];
            if (heavy[p] == -1 || sz[u] > sz[heavy[p]])
                heavy[p] = u;
        }
```

```
        vector<pair<int, int>> st{{root, root}};
        while (!st.empty()) {
            auto [u, head] = st.back();
            st.pop_back();
            for (int v = u; v != -1; v = heavy[v]) {
                top[v] = head;
                pos[v] = ++timer;
                for (int w : g[v])
                    if (par[w] == v && w != heavy[v])
                        st.push_back({w, w});
            }
        }
    }
    template <class F> void path(int u, int v, F visit) {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]])
                swap(u, v);
            visit(pos[top[u]], pos[u]);
            u = par[top[u]];
        }
        if (dep[u] > dep[v])
            swap(u, v);
        visit(pos[u], pos[v]);
    }
};
```

实现要点 pos 从 1 开始，子树区间是 [pos[u],pos[u]+sz[u]−1]。当前 path 仅适合加法、最大值等不依赖方向的聚合；字符串拼接或矩阵乘法要保留方向。边权挂在较深端点，同链最后一段排除 LCA。

8.4 换根 DP 与树重心

当题目要求分别以每个节点为根计算答案时，可先固定一个根求出子树贡献，再推导答案跨过一条边后的增量。距离和是这一方法的基本模型。

设 sz[v] 为 v 子树大小，从父 u 把根移到孩子 v，子树内 sz[v] 个点距离都少 1，其他 n−sz[v] 个点都多 1，因此 ans[v]=ans[u]+n−2sz[v]。重心则选择删除后最大连通块最小的点，最大块为 max(n−sz[u],各孩子 sz[v])。

正确性 树上两点路径唯一，根跨过边 u−v 后每个点恰好落在这条边的一侧，距离变化只能是加一或减一。重心的最大块不超过 n/2：若存在更大块，向它移动会减小最大块，直到无法继续。

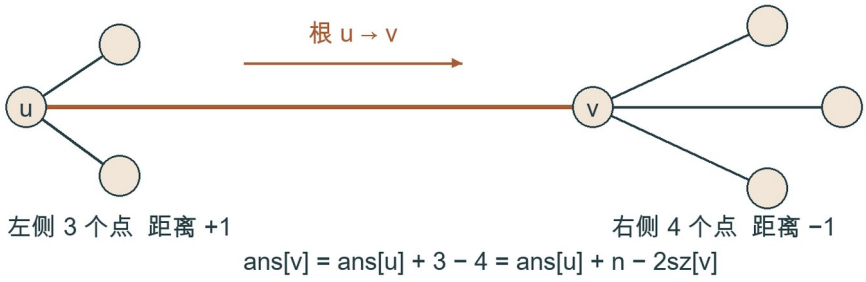


图 8.2 换根时两侧距离贡献的变化

复杂度与接口 O(n) 时间与空间。加权边下换根增量乘该边权；点带权时把节点数替换成子树权和。

```
vector<ll> all_distance_sum(const vector<vector<int>> &g) {
    int n = g.size();
    vector<int> par(n, -1), dep(n), sz(n, 1), order{0};
    par[0] = 0;
    for (int i = 0; i < n; ++i) {
        int u = order[i];
        for (int v : g[u])
            if (v != par[u]) {
                par[v] = u;
                dep[v] = dep[u] + 1;
                order.push_back(v);
            }
    }
    for (int i = n - 1; i; --i)
        sz[par[order[i]]] += sz[order[i]];
    vector<ll> ans(n);
    ans[0] = accumulate(dep.begin(), dep.end(), 0LL);
    for (int i = 1; i < n; ++i) {
        int v = order[i];
        ans[v] = ans[par[v]] + n - 2 * sz[v];
    }
    return ans;
}
```

实现要点 树直径端点、重心、中心是不同概念：中心最小化最远距离，重心控制删除后的分量大小。相同的“最优位置”题面必须先确认目标函数，不能混用结论。

8.5 树结构与边界处理

二叉树的遍历、序列化、重建与搜索树判定都应先明确节点关系及空节点的表示。一般树则通过孩子列表遍历。计算含空位的层宽时，应先规范化层内编号，避免编号随深度指数增长。

树直径在非负边权下可两次最远搜索，直径端点能帮助计算每个节点的最远距离。无权树的重心、中心与直径中点分别服务于分量控制与距离目标。树上背包合并孩子时应计入实际子树大小，避免每个点都跑满容量造成不必要的开销。

8.6 典型题解析

例题 8.1 路径与子树的统一维护

洛谷 P3384 基础验证

问题 在有根树上支持路径加、路径和、子树加与子树和，所有结果对给定模数取模。

解法 重链剖分建立 pos。子树操作对应连续区间；路径用链顶跳跃拆区间。区间数据结构使用支持加法懒标记的线段树，并将 sum 与 lazy 都按题目模数规范化。

正确性与复杂度 子树 DFS 连续性保证一段覆盖全部后代，重链拆分保证路径所有节点恰被覆盖一次。每段区间 O(log n)，路径最多 O(log n) 段，所以单次路径操作 O($\log^2 n$)。

示例推演 树边 1−2、1−3、2−4。路径 4→3 包含 4、2、1、3；子树 2 只含 2、4。两种操作不能用同一个随意截取的原始编号区间。

实现要点 本章普通 LazySum 不带取模，提交本题要调整节点更新与合并。点权路径包含 LCA，边权版本则排除 LCA 的位置。

练习建议 用小树直接枚举路径节点，对随机四种操作连续核对。

原题与解法依据 洛谷 P3384 基础验证

例题 8.2 Binary Tree 交互询问与树重心

2024 ICPC 南京 G

问题 二叉树有一个隐藏节点 s，询问 u、v 得到它们到 s 距离的大小关系；最多 $\lfloor \log_2 n \rfloor$ 次找出 s。

解法 在当前候选连通子树中求重心 c，使删除 c 后每个分量≤当前候选数的一半。

度 1 情形比较 c 与邻点。度 2 情形比较两个邻点，距离相同则 s=c，否则保留较近邻点所在分量。

度 3 时，比较较大两个分量对应的邻点；谁较近就保留其分量，相等则保留重心及第三个较小分量。

重复在缩小后的候选子树里求重心。原树的“二叉”对应无向度数≤3，不能只看根的孩子数。

正确性 两个不同分量的邻点到第三分量中任意目标的距离相同；到本分量中目标时，本侧邻点比另一侧近 2。重心保证前两个候选分量≤一半；度 3 时最小分量加重心也≤一半，可按奇偶反证。每次候选规模至少减半。

示例推演 重心两侧分量大小 4、3、2，另加重心共 10。问前两侧邻点：第一侧更近则只剩 4 个；相等则只剩重心加第三侧 3 个。

实现要点 必须控制询问数量，而不是把 BFS 距离算得更快。每次输出询问后 flush，读到失败返回值或 EOF 立即退出。本题的缩小候选区操作要连同排除标记更新。

复杂度 朴素每轮在线性候选规模内找重心，规模几何下降，总 O(n)；询问≤⌊log₂n⌋。

练习建议 用本地裁判枚举每个隐藏点，不联网提交，验证回答始终正确、询问不超过限制；先学“候选集合”而不是直接背质询代码。

原题与解法依据 2024 ICPC 南京 G

例题 8.3 The Hanged Man 树边路径覆盖与构造

2024 CCPC 济南 I

问题 给一棵树加边，要求最终每条边恰好在一个简单环中，且无自环重边。

解法 把树边划分为互不共边的路径，每条至少两条边；为每条路径的两个端点加入一条边，就闭合成一个简单环。先找任意偶度数点作为根。

DFS 从孩子处接收一条尚未闭合链的远端；若孩子内部已经全部配对，则将孩子自己视为远端，此时与当前点之间的边就是一条长度一的链。

在当前点把来自不同孩子的链两两配对，并记录两条链远端之间的新边。若孩子数为奇数，留下一条链，将它的远端传给父亲；若孩子数为偶数，全部配完，向父亲返回当前点。

根的孩子数等于偶度数，因此最后没有剩余边。若所有点度数均为奇数，则每个非根点孩子数为偶数，全部配完后只返回自身；根最后只能留下长度一的链，无法合法闭合，故无解。n=1 没有边，空加边方案合法。

正确性 每条父子边进入一条待配链，链被配对后不再返回，因而每条原边恰被覆盖一次。配对来自不同孩子的链，长度至少二，新增边不会是已有父子边。不同路径若共享两个点，就会共享这两个点之间的树路径，与边不重复矛盾；因此闭合后的环至多共享一个点，每条边仍恰在一个简单环中。

示例推演 路径 1−2−3 有偶度数节点 2，连接 1−3 后形成三角形，每条边恰在一个环。只有一条边的树两端均为奇度，若加端点连边会产生重边，因此无解。

实现要点 配对的是链的远端，不是直接配孩子编号。输出的新边无需形成单一大环。长链 n 达三十万时应使用迭代后序或明确保证递归栈容量。

复杂度 O(n) 时间和空间。

练习建议 对 n≤8 的树用暴力检查可行性；构造后检查每条树边所属环数。

原题与解法依据 2024 CCPC 济南 I

第 9 章 字符串匹配与回文结构

字符串查询先按对象分类：单模式匹配、前后缀关系、多模式统计、子串相等与回文分别使用不同结构。把匹配进度表示为自动机状态后，还可与数位 DP、图搜索和计数模型组合。

9.1 KMP 前缀函数与匹配

π[i] 是 s[0..i] 的最长真前后缀长度。失配时跳到 π[j−1]，因为任何仍可能匹配的更短前缀必须是当前前缀的 border。每次增长与回退的总次数由 j 的变化量控制为线性。



图 9.1 沿前缀函数链枚举真前后缀

复杂度与接口 前缀函数 O(m)，匹配 O(n+m)，空间 O(m)。

```
vector<int> prefix_function(const string &s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); i++) {
        int j = pi[i - 1];
        while (j && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            ++j;
        pi[i] = j;
    }
    return pi;
}
vector<int> kmp(const string &s, const string &p) {
    if (p.empty())
        return {};
    auto pi = prefix_function(p);
    vector<int> ans;
    int j = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        while (j && s[i] != p[j])
            j = pi[j - 1];
        if (s[i] == p[j])
            ++j;
        if (j == (int)p.size())
            ans.push_back(i - j + 1), j = pi[j - 1];
    }
    return ans;
}
```

实现要点 空模式单独处理。模式匹配成功后回到 pi[m−1]，才能统计重叠匹配；树与链表匹配的分叉情形不能直接照搬单串线性复杂度结论。

不断沿 π 链回退能枚举全部 border，周期长度候选为 n−π[n−1]。若要求整个串由某段重复，还需 n 能被该长度整除；只有局部周期时不应加这个条件。

树的子结构匹配若通过序列化转为字符串匹配，必须保留空节点标记。删除重复子串或计数不含某个禁串的字符串时，可把 KMP 的匹配前缀长度作为状态。

9.2 Z 函数与前缀匹配

Z[i] 表示 s[i..] 与 s 的最长公共前缀长度。前缀函数适合描述真前后缀链，Z 函数适合描述每个起点与全串前缀的匹配长度；二者都通过复用已知区间达到线性时间。

维护最右的匹配区间 [l,r]。i 在区间内时，可以复用 z[i−l]，但最多复用到 r；区间外从零开始比较。只有越过已知右端点的比较才可能推进 r，因此成功扩展的总次数为线性。

正确性 区间内的字符与前缀对应相等，镜像位置提供确定的匹配下界；超过 r 的部分尚未知，必须实际比较。每次失败只增加常数工作，每次成功越界都使右端点向右移动。

复杂度与接口 O(n) 时间、O(n) 空间，约定 z[0]=n。模式匹配可对 p+分隔符+t 求 Z，分隔符必须不在输入字符集中。

```
vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n);
    if (n)
        z[0] = n;
    for (int i = 1, l = 0, r = -1; i < n; ++i) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}
```

实现要点 空串返回空数组。不同书对 z[0] 的约定不同，调用时保持一致。拼接后的坐标需要减去模式长度与分隔符，不能直接输出原下标。

9.3 双模字符串哈希

多项式哈希把相同长度子串映射到可快速提取的指纹。比较前先统一位置归一化与长度。双模降低碰撞风险，却仍不是确定性的字符串相等证明。

复杂度与接口 构建 O(n)，子串比较 O(1)，二分 LCP O(log n)。

```
struct Hash {
    static constexpr ll P1 = 1000000007, P2 = 1000000009;
    ll B;
    vector<ll> h1, h2, p1, p2;
    Hash(const string &s, ll base)
```

```
        : B(base), h1(s.size() + 1), h2(s.size() + 1), p1(s.size() + 1, 1),
          p2(s.size() + 1, 1) {
        assert(1 < base && base < P1);
        for (int i = 0; i < (int)s.size(); i++) {
            int x = (unsigned char)s[i] + 1;
            h1[i + 1] = (h1[i] * B + x) % P1;
            h2[i + 1] = (h2[i] * B + x) % P2;
            p1[i + 1] = p1[i] * B % P1;
            p2[i + 1] = p2[i] * B % P2;
        }
    }
    pair<ll, ll> get(int l, int r) const {
        return {(h1[r] - h1[l] * p1[r - l] % P1 + P1) % P1,
                (h2[r] - h2[l] * p2[r - l] % P2 + P2) % P2};
    }
};
```

实现要点 实际比赛建议随机基数并保护对抗数据。判断需要绝对正确时优先 KMP 或确定性字符串结构；哈希匹配还必须检查长度。

面对对抗数据，可随机选择合法底数或在可疑结果上验证原串；要求严格确定性时优先 KMP、Z 或后缀工具。不要把浮点哈希或有符号溢出当作模运算。

重复字符串匹配可扩展足够次数后检查；等长单词串联可按单词长度分组维护词频窗口。近似匹配允许至多 k 处差异时，可用哈希二分下一处差异并跳过，单个起点需要 O((k+1)log n) 次量级操作，不能忽略 k。

9.4 字典树的词频与前缀频次

Trie 共享词前缀，每个节点代表一个前缀。终止计数回答完整单词频次，路径通过计数回答前缀频次；两者不能互相替代。

复杂度与接口 一次词操作 O(词长)，空间 O(总字符数×字母表大小)。

```
struct Trie {
    struct N {
        array<int, 26> ch{};
        int pass = 0, end = 0;
    };
    vector<N> t{N{}};
    void add(const string &s, int delta = 1) {
        int u = 0;
        t[u].pass += delta;
        for (char c : s) {
            int x = c - 'a';
            if (!t[u].ch[x]) {
                assert(delta > 0);
                t[u].ch[x] = t.size();
                t.emplace_back();
            }
            u = t[u].ch[x];
            t[u].pass += delta;
        }
        t[u].end += delta;
    }
    int walk(const string &s) const {
```

```
        int u = 0;
        for (char c : s) {
            u = t[u].ch[c - 'a'];
            if (!u)
                return -1;
        }
        return u;
    }
    int count(const string &s) const {
        int u = walk(s);
        return u < 0 ? 0 : t[u].end;
    }
    int prefix(const string &s) const {
        int u = walk(s);
        return u < 0 ? 0 : t[u].pass;
    }
    bool erase(const string &s) {
        if (!count(s))
            return false;
        add(s, -1);
        return true;
    }
};
```

实现要点 删除前检查单词存在。编码数字序列必须保留分隔符和符号，否则 [1,23] 与 [12,3] 可能相同。二维搜索要防止重复使用格子。

网格多词搜索可把词典放入 Trie，回溯时只有仍在 Trie 路径上才继续。多模式文本匹配则要进一步补 fail 指针形成 AC 自动机。

9.5 AC 自动机与多模式统计

模式串总长度很大时，逐个 KMP 会反复扫描文本。AC 自动机把 Trie 上的前缀共享，再用 fail 指针在匹配失败时跳到最长可用后缀。

BFS 构造 fail，同时补齐缺失转移。扫描文本时只在当前状态加一次访问计数；结束后按 BFS 逆序把计数累加给 fail 父亲。这样到达长模式的次数也计入作为其后缀的短模式，避免每个字符沿 fail 链逐个计数。

正确性 状态表示当前文本前缀的最长模式前缀后缀。一个模式出现，当且仅当当前状态的 fail 祖先链包含它的终止节点。逆序汇总正是对这条祖先关系做子树贡献合并。重复模式保存相同终点，分别读取相同计数即可。

复杂度与接口 字母表大小 σ 时建自动机 O(σL)，扫描与汇总 O(|S|+L)，空间 O(σL)。代码字符集为小写英文字母，build 后不可继续插入。

```
struct Aho {
    struct Node {
        array<int, 26> go{};
        int fail = 0;
        ll cnt = 0;
    };
    vector<Node> t{Node{}};
    vector<int> order;
    int insert(const string &s) {
```

```
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!t[u].go[x]) {
                t[u].go[x] = t.size();
                t.push_back(Node{});
            }
            u = t[u].go[x];
        }
        return u;
    }
    void build() {
        queue<int> q;
        for (int v : t[0].go)
            if (v)
                q.push(v);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int x = 0; x < 26; ++x) {
                int v = t[u].go[x];
                if (v) {
                    t[v].fail = t[t[u].fail].go[x];
                    q.push(v);
                } else
                    t[u].go[x] = t[t[u].fail].go[x];
            }
        }
    }
    void count(const string &s) {
        for (auto &v : t)
            v.cnt = 0;
        int u = 0;
        for (char c : s) {
            u = t[u].go[c - 'a'];
            ++t[u].cnt;
        }
        for (int i = int(order.size()) - 1; i >= 0; --i) {
            int v = order[i];
            t[t[v].fail].cnt += t[v].cnt;
        }
    }
};
```

实现要点 count 已清空旧计数，可重复统计不同文本。空模式不在本模板契约内；字符范围必须检查。使用“已访问标记后把终点清零”的简单版，只适用于求出现过的模式数量，不适合求每个模式的全部出现次数。

9.6 二进制 Trie 与最大异或

最大异或从最高位决定胜负：在已固定更高位后，当前位尽量选与 x 相反的分支。如果该分支存在任何候选，就一定优于同位分支，不需考虑更低位补偿。

复杂度与接口 最大异或 O(nB)，空间 O(nB)，B 为二进制位数。

```
struct XorTrie {
    vector<array<int, 2>> t{{-1, -1}};
    void add(uint32_t x) {
        int u = 0;
        for (int b = 31; b >= 0; b--) {
            int v = x >> b & 1;
            if (t[u][v] < 0) {
                t[u][v] = t.size();
                t.push_back({-1, -1});
            }
            u = t[u][v];
        }
    }
    uint32_t best(uint32_t x) const {
        int u = 0;
        uint32_t ans = 0;
        for (int b = 31; b >= 0; b--) {
            int v = x >> b & 1;
            if (t[u][v ^ 1] >= 0)
                ans |= 1U << b, u = t[u][v ^ 1];
            else
                u = t[u][v];
            assert(u >= 0);
        }
        return ans;
    }
};
```

实现要点 本实现处理 $[0,2^{32})$；需要 64 位时调整 B 和类型。至少插入一个数才能查询；求数对时先查询再插入，避免误把自身当成之前的元素。

区间或滑动窗口异或需要支持删除与节点计数，静态只插入模板不能直接沿已空分支走。带上界条件时还需维护节点索引或按查询阈值离线插入。

9.7 Manacher 与全部回文半径

最长回文子串不能由最长回文子序列 DP 替代。子串要求连续，Manacher 利用左右对称，把所有中心的扩展复用成线性计算。分别存奇、偶回文半径可以避免引入特殊分隔字符。

d1[i] 表示以 i 为中心的奇回文半径，长度为 2d1[i]−1；d2[i] 表示中心位于 i−1 和 i 之间的偶回文半径，长度为 2d2[i]。维护最右回文区间，区间内部用对称中心的已知半径初始化，再尝试越界扩展。

正确性 镜像回文在已知区间内必然成立，超出右边界后才需要验证。与 Z 函数相同，右边界只前进，故总扩展次数为 O(n)。所有半径之和还等于回文子串的数量。

复杂度与接口 O(n) 时间和空间；返回最长回文子串长度，空串为零。

```
int longest_palindrome(const string &s) {
    int n = s.size(), ans = 0;
    vector<int> d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k])
```

```
            ++k;
        d1[i] = k;
        ans = max(ans, 2 * k - 1);
        if (i + k - 1 > r)
            l = i - k + 1, r = i + k - 1;
    }
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k])
            ++k;
        d2[i] = k;
        ans = max(ans, 2 * k);
        if (i + k - 1 > r)
            l = i - k, r = i + k - 1;
    }
    return ans;
}
```

实现要点 奇偶中心定义不同。若要求输出区间，用中心和半径换算起点；若求所有回文出现次数，累计用 ll，不能只累加最大长度。

9.8 解析与自动机组合

嵌套字符串、括号表达式和分子式都可用递归下降：每层函数只处理本层语法，遇到开括号调用下一层，遇到闭括号返回。运算优先级分成表达式、项与原子三层，数字可能多位，空白与合法性按题面处理。

Trie 的节点或 KMP 的匹配长度都能作为 DP 自动机状态。禁串计数先构造转移，再与长度、余数、数位边界组合；到达完整禁串状态即拒绝。哈希适合频繁子串比较，自动机适合推进与计数，不能仅按模板长度选择。

9.9 典型题解析

例题 9.1 所有模式串的出现次数

洛谷 P5357 基础验证

问题 给出多个非空小写模式串和一个长文本，分别统计每个模式出现次数，允许重复模式。

解法 插入每个模式并保存终止节点。构建 AC 自动机，文本扫描只对当前状态 cnt++。按 BFS 逆序向 fail 父亲累加，最后每个模式读终止节点计数。

正确性与复杂度 某短模式是当前状态的后缀时，其终点必在 fail 链上。逆序累加将每次匹配贡献传播到所有这类后缀，且不会跨越错误的后缀关系。建图 O(26L)，扫描 O(|S|+L)。

示例推演 模式 a、aa、a，文本 aaa。长状态出现也贡献给 a，三项答案为 3、2、3；重复模式应分别输出，不能被去重后漏一项。

实现要点 按文本字符逐个遍历 fail 链，最坏可退化。也不能使用“每个模式首次匹配后清零”的计数方法。

练习建议 与逐模式朴素匹配对拍，测试包含关系、重复模式和大量相同字符。

原题与解法依据 洛谷 P5357 基础验证

例题 9.2 最长回文子串与半径复用

洛谷 P3805 基础验证

问题 求字符串中最长连续回文片段的长度，字符串可以非常长。

解法 同时计算 d1 与 d2。最右回文内部复用镜像半径，越过已知右端后实际比较。分别取 max(2d1−1) 与 max(2d2)。

正确性与复杂度 镜像保证区间内匹配，只有推进最右端的比较会产生大量成功扩展，右端最多移动 n 次。因此总时间 O(n)、空间 O(n)。

示例推演 abba 的答案来自偶半径 d2[2]=2；aba 来自奇半径 d1[1]=2。两者都输出实际原串长度，不能把半径当长度。

实现要点 最长回文子序列可跳过字符，答案和复杂度都不同。处理一千万字符时也要计算两个 int 数组与文本的总内存。

练习建议 与枚举所有子串的回文检查对拍，再加入奇偶混合字符串。

原题与解法依据 洛谷 P3805 基础验证

例题 9.3 Pen Pineapple Apple Pen KMP 的 border 计数

2025 ICPC 南京 H

问题 从串中按顺序选 4 个不重叠非空子串，要求第一与第四相同，第三是第二的严格后缀，统计方案数。

解法 约定字符串从 1 开始。f[L,R] 表示第一段在 L 结束、第四段从 R 开始的相等串选择数；g[l,r] 表示第二、三段都在 [l,r] 内的选择数。答案为 Σ f[l−1,r+1]·g[l,r]，求和范围 2≤l≤r<n。两侧边界固定后，内外选择独立。

固定 R，以 P=S[R..n] 为模式串。先求 π 和 depth[t]=1+depth[π[t−1]]，depth[0]=0。再扫描 S[1..n]，若在 L 结束的最长匹配长度为 q，则 f[L,R]=depth[q]：q 自身与失配链上的所有正长度，都给出一个相等外侧串。

定义 h[i,j] 为 S[i..j] 中长度不超过区间长度一半的正 border 数。每个这样的 border 指定第二段的后缀与第三段，第二段起点可以选在 l..i−1，因此有 i−l 种。故 g[l,r]=Σ(l≤i≤j≤r)(i−l)h[i,j]。

h 用逐后缀 KMP 求得。设当前区间长度为 m、最长 border 为 b。若 b>⌊m/2⌋，最短周期 p=m−b；超过一半的 border 依次减 p，用整除直接跳到最后一个长 border，再沿 π 跳一次到合法长度。h 等于该合法长度对应的 depth。重叠 border 强制周期，因此长 border 链可以成段跳过。

对 h[i,j] 与 i·h[i,j] 分别建立二维前缀，j<i 的位置置零。查询正方形 [l,r]×[l,r]，得到 A=Σh、B=Σi·h，于是 g=B−lA。所有计算对 998244353 取模，减法规范为非负。

正确性 f 的匹配失配链恰好枚举全部相等外侧串。h 的半长限制保证两段后缀不重叠，权重 i−l 保证第二段比第三段严格更长。二维前缀只将同一组合法选择加速汇总；固定中间覆盖边界后，每组四段恰好计数一次。逐后缀 KMP、表填充和全部边界求和均为 $O(n^2)$。

示例推演 对于串 ababa，border 长度为 3 和 1；长度 3 超过串长一半，若两段要不重叠则不能选它。沿失配链跳到 1 后才符合长度限制。

实现要点 border 必须为真前后缀，h 不包含零长度。f 的 depth 需要把最长匹配自身算一次，h 则从最长真 border 开始。KMP 扫描达到整个模式长度后应保留当次答案，再按失配链继续后续字符。二次数组在 n=5000 时占用显著，按模整数存储并估算总表数量。

复杂度 命题组解法 $O(n^2)$；表按实现取 $O(n^2)$ 空间。

练习建议 先做 border 个数和长度≤一半的 border 查询，再完成两张计数表。对长度≤10 的串枚举四段端点验证。

原题与解法依据 2025 ICPC 南京 H

第 10 章 数论组合与高精度计算

数论与组合题应先确定计数对象、整除关系和数值范围。模除法要求逆元存在，组合恒等式要求模型匹配，高精度运算则需同时估计位数与操作次数。

10.1 模乘快速幂与矩阵快速幂

快速幂把指数按二进制拆分，每次平方一次底数。矩阵乘法同样满足结合律，所以用转移矩阵把线性递推推到第 n 项。行向量或列向量只要从定义到乘法始终一致即可。

复杂度与接口 标量 O(log e)，d 阶矩阵 O(d^3 log e)。

```
ll modpow(ll a, ll e, ll mod) {
    assert(e >= 0 && mod > 0);
    a = (ll)(((i128)a % mod + mod) % mod);
    ll r = 1 % mod;
    while (e) {
        if (e & 1)
            r = (i128)r * a % mod;
        a = (i128)a * a % mod;
        e >>= 1;
    }
    return r;
}
using Mat = vector<vector<ll>>;
Mat multiply(const Mat &a, const Mat &b, ll mod) {
    int n = a.size();
    Mat c(n, vector<ll>(n));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++)
            for (int j = 0; j < n; j++)
                c[i][j] = (c[i][j] + (i128)a[i][k] * b[k][j]) % mod;
    return c;
}
Mat matpow(Mat a, ll e, ll mod) {
    assert(e >= 0);
    int n = a.size();
    Mat r(n, vector<ll>(n));
    for (int i = 0; i < n; i++)
        r[i][i] = 1 % mod;
    while (e) {
        if (e & 1)
            r = multiply(r, a, mod);
        a = multiply(a, a, mod);
        e >>= 1;
    }
    return r;
}
```

实现要点 模数为正，指数非负。逆元 a^(p−2) 要求 p 为质数且 a%p≠0。模乘支持正的 64 位有符号模数。

矩阵规模 d 的普通乘法为 $O(d^3)$，快速幂 $O(d^3 \log n)$，不能只写 $O(\log n)$。模乘使用提升类型处理乘积，仍须确保提升后的类型能覆盖最大乘积。

转移矩阵可表示 Fibonacci、铺砖、元音相邻限制、出勤记录与图上恰好 k 步的游走计数。游走允许重复顶点；至多 k 步的计数可增加累计状态或求矩阵等比和。矩阵元素均应规范到模数范围。

10.2 素数筛素性检验与因数分解

试除只需到 $\sqrt{n}$，因为大因数与小因数成对。埃氏筛从素数 p 的 p^2 起标记倍数，O(NloglogN)；欧拉线性筛让每个合数只由最小素因子产生一次，O(N)。重复小值域查询先筛，少量超大输入再用素性检验。

对奇数 n 写 $n-1=d\cdot 2^s$，d 为奇数。素数时，对任意非零底数 a，a^d 要么等于 1，要么反复平方会在进入 1 之前出现 −1。违背该条件即可判合数；通过单一底数仍可能是强伪素数。本节使用覆盖 64 位范围的固定底数组合，并以 128 位乘积保证模乘精确。

复杂度与接口 线性筛 O(V)，试除 O(√x)；Miller Rabin 使用七组固定底数，固定宽度模运算下需 O(log x) 次量级运算。

```
struct Sieve {
    vector<int> spf, primes;
    explicit Sieve(int n) : spf(n + 1) {
        for (int i = 2; i <= n; i++) {
            if (!spf[i])
                spf[i] = i, primes.push_back(i);
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > n)
                    break;
                spf[i * p] = p;
            }
        }
    }
    vector<pair<int, int>> factor(int x) const {
        vector<pair<int, int>> r;
        while (x > 1) {
            int p = spf[x], c = 0;
            do {
                x /= p;
                ++c;
            } while (x > 1 && spf[x] == p);
            r.push_back({p, c});
        }
        return r;
    }
};
using u64 = unsigned long long;
u64 mul64(u64 a, u64 b, u64 m) {
    return (__uint128_t)a * b % m;
}
u64 pow64(u64 a, u64 e, u64 m) {
    u64 r = 1;
    while (e) {
        if (e & 1)
```

```
            r = mul64(r, a, m);
        a = mul64(a, a, m);
        e >>= 1;
    }
    return r;
}
bool miller_rabin(u64 n) {
    if (n < 2)
        return false;
    for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
        if (n % p == 0)
            return n == p;
    u64 d = n - 1;
    int s = 0;
    while (!(d & 1))
        d >>= 1, ++s;
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL,
                  1795265022ULL}) {
        if (a % n == 0)
            continue;
        u64 x = pow64(a % n, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool pass = false;
        for (int r = 1; r < s; ++r) {
            x = mul64(x, x, n);
            if (x == n - 1) {
                pass = true;
                break;
            }
        }
        if (!pass)
            return false;
    }
    return true;
}
```

实现要点 本节 Miller Rabin 的七组底数覆盖无符号 64 位整数，模乘使用 unsigned __int128。超出 64 位范围不能沿用这一保证；判断素数与分解质因数是不同任务。

因数枚举得到全部约数，质因数分解得到素因子及其指数，二者应按后续公式的需要选择。筛法适合可预处理的值域；大整数分解另需专门算法。

第 n 个被 a 或 b 整除的数，可用 x/a+x/b−x/lcm(a,b) 计数后进行二分。质因子连通可让共享素因子的数进入同一分量，贡献计数应按素因子指数或出现位置组织。大整数因数分解需要不同工具，线性筛不自动覆盖 Pollard Rho。

64 位确定性 Miller Rabin 底数记录

10.3 欧拉函数与莫比乌斯函数

欧拉函数 φ(n) 数 1..n 中与 n 互质的数；莫比乌斯函数 μ(n) 用于把“最大公约数恰为某值”转为整除条件的容斥。二者可以在线性筛中一起计算。

对新素数 p，φ(p)=p−1、μ(p)=−1。若 p 整除 i，则 φ(ip)=pφ(i)、μ(ip)=0，并停止筛该 i 的后续素数；否则 φ(ip)=(p−1)φ(i)、μ(ip)=−μ(i)。互质点对数可写成 Σμ(d)⌊n/d⌋⌊m/d⌋。

正确性 每个合数只由其最小素因子产生一次。莫比乌斯恒等式 Σ(d|x)μ(d) 等于 x=1 时的 1、其他时候的 0，因此按公因子展开后，只有互质点对被保留。

复杂度与接口 预处理 O(N) 时间和空间；互质对直接求和 O(min(n,m))，配合整除分块可进一步减少求和次数。

```
struct ArithmeticSieve {
    vector<int> primes, phi, mu, composite;
    ArithmeticSieve(int n) : phi(n + 1), mu(n + 1), composite(n + 1) {
        if (n >= 1)
            phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!composite[i]) {
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : primes) {
                if (p > n / i)
                    break;
                int x = i * p;
                composite[x] = 1;
                if (i % p == 0) {
                    phi[x] = phi[i] * p;
                    mu[x] = 0;
                    break;
                }
                phi[x] = phi[i] * (p - 1);
                mu[x] = -mu[i];
            }
        }
    }
};
```

实现要点 μ 不是所有数论题都应使用的工具。先尝试按 gcd 分类、因数枚举和普通容斥；式子中存在整除计数且需要筛掉非互质时，再考虑反演。

10.4 扩展欧几里得与扩展 CRT

exgcd 求 ax+by=gcd(a,b)。线性同余 ax≡c mod m 有解当且仅当 gcd(a,m) 整除 c。合并 x≡r1 mod m1 与 x≡r2 mod m2 时，对 r2−r1 求线性同余，非互质模数也可处理。

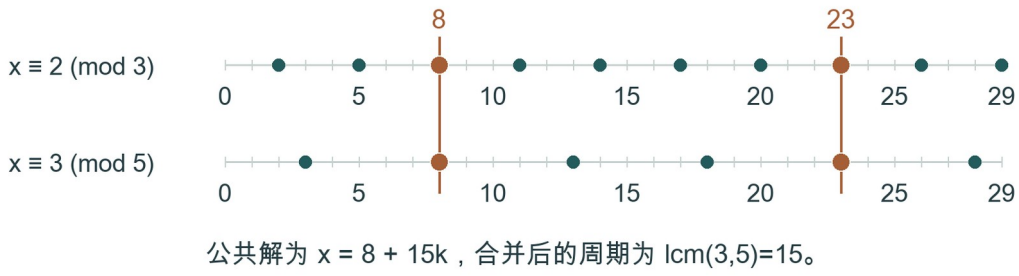


图 10.1 同余方程的公共解与合并周期

复杂度与接口 一组方程 O(log V)，多组依次合并；本实现 i128 保存中间乘积。

```
i128 exgcd(i128 a, i128 b, i128 &x, i128 &y) {
    if (!b) {
        x = 1;
        y = 0;
        return a;
    }
    i128 u, v, g = exgcd(b, a % b, u, v);
    x = v;
    y = u - a / b * v;
    return g;
}
i128 norm(i128 x, i128 m) {
    return (x % m + m) % m;
}
optional<pair<i128, i128>> crt(i128 a, i128 m, i128 b, i128 n) {
    assert(m > 0 && n > 0);
    a = norm(a, m);
    b = norm(b, n);
    i128 x, y, g = exgcd(m, n, x, y), d = b - a;
    if (d % g)
        return nullopt;
    i128 step = n / g, t = norm(d / g * x, step);
    i128 period = m * step;
    return pair<i128, i128>{norm(a + m * t, period), period};
}
```

实现要点 必须规范负余数。CRT 新周期可能超出 long long，接口用 i128；超过 i128 时仍需高精度。扩展 gcd 接口要求参数非负。

每次合并的新模数是最小公倍数，乘积可能迅速增大。无解和模数超过类型范围是不同情况，不能都返回某个普通余数。合并后将余数规范到 [0,m)。

格点线段上的整点数为 gcd(|dx|,|dy|)+1。简单格点多边形满足 Pick 定理：面积=内部整点数+边界整点数/2−1。顶点必须是整点，多边形必须简单且不自交。

10.5 组合数阶乘逆元与 Catalan

组合数阶乘逆元法利用 C(n,k)=n!/(k!(n−k)!)，除法在素数模数下用逆元替代。所有阶乘非零要求 n<p。Catalan 的除以 n+1 同样需要逆元存在。

复杂度与接口 预处理 O(N+log p)，查询 O(1)，空间 O(N)。

```
struct Comb {
    ll p;
    vector<ll> f, iv;
    Comb(int N, ll prime) : p(prime), f(N + 1, 1), iv(N + 1, 1) {
        assert(N < p); // prime 的素性由调用者保证。
        for (int i = 1; i <= N; i++)
            f[i] = (i128)f[i - 1] * i % p;
        iv[N] = modpow(f[N], p - 2, p);
        for (int i = N; i > 0; i--)
            iv[i - 1] = (i128)iv[i] * i % p;
    }
    ll C(int n, int k) const {
        if (k < 0 || k > n)
            return 0;
        return (i128)f[n] * iv[k] % p * iv[n - k] % p;
    }
};
ll catalan_dp(int n, ll mod) {
    vector<ll> c(n + 1);
    c[0] = 1 % mod;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j < i; j++)
            c[i] = (c[i] + (i128)c[j] * c[i - 1 - j]) % mod;
    return c[n];
}
```

实现要点 模数不是质数、或 n≥p 时，阶乘逆元板子失效。Catalan 的卷积递推不需要除法，可处理任意正模数，但时间 $O(n^2)$。

如果 p 整除 n+1，不能直接乘 (n+1) 的逆元。可改用 C(2n,n)−C(2n,n+1) 等无需这一步除法的表达，并根据大 n 选择 Lucas 或其他方法。

Catalan 数常对应合法括号、合法出栈序、非交叉配对、二叉树形状与凸多边形三角剖分。使用前应建立到这些对象的双射，或推导同一递推；仅出现“分成左右两部分”并不足以套用。

10.6 Lucas 定理与整除分块

阶乘逆元法要求预处理范围小于素数模数。n 很大而模数 p 是较小素数时，可以按 p 进制逐位计算组合数。另一种常见压缩来自 ⌊n/i⌋：它只在 $O(\sqrt{n})$ 个区间上变化。

Lucas 公式为 $C(n,k)\equiv\prod C(n_i,k_i) \bmod p$。每位小于 p，局部组合数用阶乘表即可。整除分块中，从左端 l 出发，商 q=⌊n/l⌋ 不变的最右位置是 r=⌊n/q⌋；对区间求和后跳到 r+1。

正确性 在模 p 下 (1+x)^p≡1+x^p，按 p 进制展开 (1+x)^n，各位系数相乘得到 Lucas。分块的右端由商的定义精确反解，覆盖完整区间且不重不漏。

复杂度与接口 Lucas 预处理 O(p)，每次 O(log_p n)；要求 p 为素数且能容纳长度 p 的表。分块求 Σ⌊n/i⌋ 为 $O(\sqrt{n})$。

```
ll lucas(ll n, ll k, ll p, const vector<ll> &fac, const vector<ll> &ifac) {
    ll ans = 1;
    while (n || k) {
```

```
        ll a = n % p, b = k % p;
        if (b > a)
            return 0;
        ans = ans * fac[a] % p;
        ans = ans * ifac[b] % p * ifac[a - b] % p;
        n /= p;
        k /= p;
    }
    return ans;
}
i128 floor_sum_positive(ll n) {
    i128 ans = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll q = n / l;
        r = n / q;
        ans += i128(r - l + 1) * q;
        if (r == n)
            break;
    }
    return ans;
}
```

实现要点 Lucas 不适用于任意合数模数。p≈10^9 时不能开长度 p 的阶乘表。乘法范围由 p 决定，本实现要求两数模乘不超 ll；更大模数使用安全模乘。

10.7 容斥与二项式反演

容斥先计算容易统计的“至少满足某些条件”，按选择条件数交替加减，消除重复计数。二项式反演把“至少 k 项”与“恰好 k 项”这两种计数关系互相转换。

复杂度与接口 子集容斥 $O(2^k)$，倍数统计常为 $O(V \log V)$。

```
ll divisible_union(ll N, const vector<ll> &primes) {
    int k = primes.size();
    assert(k < 25);
    ll ans = 0;
    for (int mask = 1; mask < (1 << k); mask++) {
        ll product = 1;
        bool ok = true;
        for (int i = 0; i < k; i++)
            if (mask >> i & 1) {
                if (product > N / primes[i]) {
                    ok = false;
                    break;
                }
                product *= primes[i];
            }
        if (ok)
            ans += (__builtin_popcount((unsigned)mask) & 1 ? 1 : -1) *
                   (N / product);
    }
    return ans;
}
```

实现要点 符号从集合大小奇偶决定。被多个质因子整除计数时只用互异质因子；乘积超过上界要及时停止，不能让乘法先溢出。

条件很多时指数复杂度不可接受，应利用重复、约数结构或预处理。枚举子集乘积、最小公倍数时先判断是否已超过范围，不能先溢出再做比较。

有限硬币计数先做无限完全背包，再对超限硬币子集平移目标金额容斥；不同 gcd 的子序列计数按倍数倒推；歌单限制和分特产使用“缺少哪些对象”的容斥。二项式反演：若 g(n)=Σ C(n,k)f(k)，则 f(n)=Σ (−1)^(n−k)C(n,k)g(k)。证明来自交换求和后 (1−1) 的二项式展开。

10.8 高精度整数与精确算术

固定宽度整数无法表示千位数据，浮点数也不能提供这种规模下的精确比较。评测环境支持 Boost 时，可用 cpp_int 实现任意精度的整数运算。

加减乘除沿用整数语义，读入十进制字符串可逐位累积。除法仍是整数商，需同时检查余数；比较与排序精确。cpp_int 的开销随位数增长，不能把大量大整数运算当成常数操作。

正确性 整数每一步均精确表示，进位与借位由库完成，因此只要代数推导正确，不会丢失低位信息。策略压缩应先在小整数上证明并对拍，再换类型；高精度工具不能修补错误的计价规则。

复杂度与接口 长度 D 的加减和比较通常 O(D)，乘除取决于实现与位数。适合位数大、运算次数可控的题；需确认评测编译器提供 Boost。

```
#include <boost/multiprecision/cpp_int.hpp>
using boost::multiprecision::cpp_int;
cpp_int decimal_integer(const string &s) {
    cpp_int x = 0;
    for (char c : s) {
        assert('0' <= c && c <= '9');
        x = x * 10 + c - '0';
    }
    return x;
}
// 精确正整数向上除法: a>=0, b>0
cpp_int ceil_div_big(cpp_int a, cpp_int b) {
    assert(a >= 0 && b > 0);
    return a / b + (a % b != 0);
}
```

实现要点 本模板只解析非负十进制，不接受符号。不要把高精度结果先转为 ll 再比较；需要取模时先对 cpp_int 求余，确认余数范围后再转换。

Boost Multiprecision 官方说明

10.9 常见计数公式与置换结构

错排满足 D[n]=(n−1)(D[n−1]+D[n−2])，D[0]=1、D[1]=0。康托展开把某位置右侧更小元素数乘以对应阶乘，累加得到排列排名；逆展开依次选择剩余集合的第 k 小，适合频次 Fenwick。重复元素不能直接套普通排列的阶乘进制。

置换可拆成不相交环，每个环长度决定反复置换的周期；Josephus 用 f[n]=(f[n−1]+k) mod n 求最后幸存者，约定从 0 编号。完全洗牌、三次反转和循环移位都应先描述位置映射，再推导原地操作。

格点多边形的面积可用鞋带公式，边界格点数沿每条边累加 gcd(|Δx|,|Δy|)。简单格点多边形满足 Pick 公式 S=I+B/2−1。前提包括简单、不自交与整数坐标，不能扩展到任意自交图形。

10.10 典型题解析

例题 10.1 The Chariot 超长整数与策略压缩

2024 CCPC 济南 E

问题 出租车起步 A 元含 X 米，之后 Y 米按 B 元每米，其后按 C 元每米。随时可重新打车，求 D 米最低成本。六个参数可长达约 2077 位。

解法 先固定总共使用 k 辆车。每辆起步覆盖 X 米，总起步费 kA；若 D≤kX，已经足够。否则还要付费跑 R=D−kX 米。

若 B≤C，应优先用完所有车辆较便宜的 B 区间，共 kY 米，因此费用为 kA+B·min(R,kY)+C·max(R−kY,0)。若 B>C，应把付费里程集中到同一辆车，最多经历一次 Y 米的 B 区间，再用更便宜的 C；费用为 kA+B·min(R,Y)+C·max(R−Y,0)。

只需考虑 1≤k≤⌈D/X⌉，更多车辆只会增加起步费。每一段内费用关于 k 都是一次函数，最小值在该段的整数端点取得。B≤C 时分界在 D/(X+Y)、D/X；B>C 时分界在 (D−Y)/X、D/X。

收集 1、⌈D/X⌉，以及各非负分界的向下取整与再加一；过滤超出范围的 k，用上式计算并取最小。这把命题组按 A、AB 与 C 计价段列出的有限策略，改写成统一的分段线性候选算法。所有运算使用高精度整数。

正确性 固定 k 时，免费里程总应尽量使用。B 不贵于 C 时，把付费里程分配到仍有 B 容量的车不会变差；B 贵于 C 时，将多个付费尾段合到同一车能减少反复跨越 B 区间。所得费用在各分界之间为一次函数，整数最优只能位于端点附近，有限候选完整覆盖。

示例推演 A=10、X=10、Y=10、B=1、C=5、D=25。使用一辆费用 45；两辆先付 20，剩余 5 米按 B 付 5，总费 25；三辆费用 30。最优为两辆。不能只比较 A/X、B、C 后决定一直使用某个区间。

```
// 依赖本章的 cpp_int
cpp_int taxi_cost(cpp_int A, cpp_int B, cpp_int C, cpp_int D, cpp_int X,
                  cpp_int Y) {
    cpp_int limit = (D + X - 1) / X;
    vector<cpp_int> cand{1, limit};
    auto boundary = [&](cpp_int q) {
        cand.push_back(q);
        cand.push_back(q + 1);
    };
    boundary(D / X);
    if (B <= C)
```

```
        boundary(D / (X + Y));
    else if (D > Y)
        boundary((D - Y) / X);
    cpp_int best;
    bool found = false;
    for (cpp_int k : cand) {
        if (k < 1 || k > limit)
            continue;
        cpp_int r = D - k * X, value = k * A;
        if (r > 0) {
            cpp_int cap = Y;
            if (B <= C)
                cap = k * Y;
            cpp_int z = min(r, cap);
            value += z * B + (r - z) * C;
        }
        if (!found || value < best)
            best = value, found = true;
    }
    return best;
}
```

实现要点 D≤Y 时跳过负分界，过滤 k 的范围，R 为负时截零。千位参数不能转成浮点比较均价。

复杂度 常数个候选，实际成本随高精度数码长度增长。

练习建议 先用短参数的逐米 DP 验证候选策略，再换高精度类型。

原题与解法依据 2024 CCPC 济南 E

第 11 章 线性代数概率与博弈

线性基与高斯消元要求明确运算数域，概率与期望要求明确随机后继，博弈要求明确行动方及终止规则。本章按照这些前提分别建立可复用的数学模型。

11.1 异或线性基

异或线性基把整数看作 GF2 向量，每个基向量有不同最高置位。插入时从高位消去，最终为零表示可由已有基表示。贪心最大异或也是从最高位决定是否采用基向量。

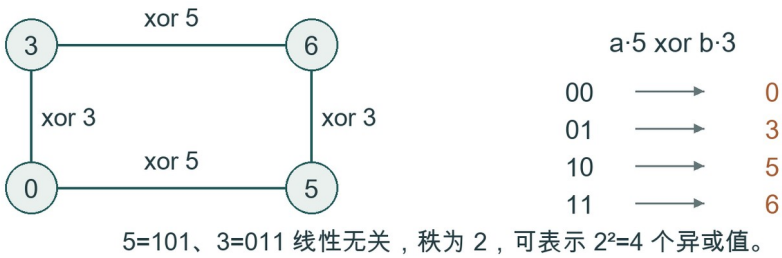


图 11.1 两个基向量生成的异或空间

复杂度与接口 插入 O(B)，最大值 O(B)，空间 O(B)。

```
struct Basis {
    array<unsigned long long, 64> b{};
    int rank = 0;
    bool add(unsigned long long x) {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1) {
                if (b[i])
                    x ^= b[i];
                else {
                    b[i] = x;
                    ++rank;
                    return true;
                }
            }
        return false;
    }
    unsigned long long maximum() const {
        unsigned long long x = 0;
        for (int i = 63; i >= 0; i--)
            x = max(x, x ^ b[i]);
        return x;
    }
    bool contains(unsigned long long x) const {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
                x ^= b[i];
        return x == 0;
    }
};
```

实现要点 原数组非空子集能否异或为 0 与 rank<n 等价；不能混淆“0 总能由空集得到”。第 k 小需要额外规范化消元，不能直接按普通插入顺序编号。

最大异或与第 k 小可表示异或不同；后者要先化成规范基，并确认是否包含零以及是否要求非空子集。存在相关向量时零也可能由非空子集得到。

平方乘积子集将每个质因子指数的奇偶转成向量，解齐次 GF(2) 方程。魔法矿石按正价值降序尝试插基是线性无关集合的贪心；若价值允许负数，必须跳过负收益而不是强求满秩。实数向量线性基采用误差判定，不能复用整数异或消元。

11.2 实数高斯消元

高斯消元把每列的主元变为一，再消去其他行对应列，从而读出变量。没有主元的列表示自由变量；全零系数但非零常数的行表示无解。

复杂度与接口 m 条方程、n 个未知数 $O(mn^2)$，方阵为 $O(n^3)$。

```
int gauss(vector<vector<double>> a, vector<double> &x) {
    const double EPS = 1e-9;
    int m = a.size(), n = a[0].size() - 1, row = 0;
    vector<int> where(n, -1);
    for (int c = 0; c < n && row < m; c++) {
        int sel = row;
        for (int i = row; i < m; i++)
            if (abs(a[i][c]) > abs(a[sel][c]))
                sel = i;
        if (abs(a[sel][c]) < EPS)
            continue;
        swap(a[sel], a[row]);
        where[c] = row;
        double t = a[row][c];
        for (int j = c; j <= n; j++)
            a[row][j] /= t;
        for (int i = 0; i < m; i++)
            if (i != row) {
                double k = a[i][c];
                for (int j = c; j <= n; j++)
                    a[i][j] -= k * a[row][j];
            }
        ++row;
    }
    for (int i = 0; i < m; i++) {
        bool zero = true;
        for (int j = 0; j < n; j++)
            zero &= abs(a[i][j]) < EPS;
        if (zero && abs(a[i][n]) >= EPS)
            return -1;
    }
    x.assign(n, 0);
    for (int j = 0; j < n; j++)
        if (where[j] >= 0)
            x[j] = a[where[j]][n];
    return row == n ? 1 : 0;
}
```

实现要点 EPS 根据数据规模选择；接口返回 −1 无解、0 多解、1 唯一解。球心问题先将两点等距方程相减，消去平方未知项。

实数版本选绝对值较大的主元降低误差，eps 不是精确零判断。无解、唯一解和无穷多解要分别返回；不能把自由变量默认为零后声称唯一。

运算数域决定消元规则。实数消元需误差判定，素数模消元使用逆元，GF(2) 消元使用异或；合数模中的非零主元可能不可逆。最少开关次数还需优化解空间，不能仅返回任意一组解。

11.3 GF2 高斯消元的位压缩版本

GF2 消元中加减都变成异或，可以用 bitset 同时消去整行。除主元归一化外，消元结构与实数相同；主元本身只有 1，不需要求逆。

复杂度与接口 O(mn²/机器字长)，空间 O(mn/机器字长)。

```
int gauss2(vector<bitset<513>> a, int n, vector<int> &x) {
    assert(n <= 512);
    int m = a.size(), r = 0;
    vector<int> w(n, -1);
    for (int c = 0; c < n && r < m; c++) {
        int s = r;
        while (s < m && !a[s][c])
            ++s;
        if (s == m)
            continue;
        swap(a[s], a[r]);
        w[c] = r;
        for (int i = 0; i < m; i++)
            if (i != r && a[i][c])
                a[i] ^= a[r];
        ++r;
    }
    for (int i = 0; i < m; i++) {
        bool zero = true;
        for (int c = 0; c < n; c++)
            zero &= !a[i][c];
        if (zero && a[i][n])
            return -1;
    }
    x.assign(n, 0);
    for (int c = 0; c < n; c++)
        if (w[c] >= 0)
            x[c] = a[w[c]][n];
    return r;
}
```

实现要点 bitset 容量是编译时常数，本实现要求 n≤512。最小操作数需要枚举自由变量，接口返回的是任意一组解。

用于开关、平方子集等题时，先明确每一列代表变量还是条件。自由变量个数 f 对应 2^f 个解；若要求最小操作数，单次任意解不是最优解。

11.4 素数模高斯消元

素数模消元在有限域中进行，每个非零主元都可以用费马逆元归一化。零判断精确，不存在 eps。每次加减和乘法后应规范模余数。

复杂度与接口 $O(mn^2+n \log p)$。

```
int gauss_mod(vector<vector<ll>> a, ll p, vector<ll> &x) {
    int m = a.size(), n = a[0].size() - 1, r = 0;
    vector<int> w(n, -1);
    for (auto &row : a)
        for (ll &v : row)
            v = (ll)(((i128)v % p + p) % p);
    for (int c = 0; c < n && r < m; c++) {
        int s = r;
        while (s < m && !a[s][c])
            ++s;
        if (s == m)
            continue;
        swap(a[s], a[r]);
        w[c] = r;
        ll iv = modpow(a[r][c], p - 2, p);
        for (int j = c; j <= n; j++)
            a[r][j] = (i128)a[r][j] * iv % p;
        for (int i = 0; i < m; i++)
            if (i != r) {
                ll k = a[i][c];
                for (int j = c; j <= n; j++)
                    a[i][j] =
                        (ll)(((i128)a[i][j] - (i128)k * a[r][j] % p + p) % p);
            }
        ++r;
    }
    for (int i = 0; i < m; i++) {
        bool zero = true;
        for (int c = 0; c < n; c++)
            zero &= a[i][c] == 0;
        if (zero && a[i][n])
            return -1;
    }
    x.assign(n, 0);
    for (int c = 0; c < n; c++)
        if (w[c] >= 0)
            x[c] = a[w[c]][n];
    return r == n ? 1 : 0;
}
```

实现要点 调用者保证 p 是质数，输入系数先归一化。合数模不能直接用 $p-2$ 次幂求主元逆元。

合数模数里非零元素也可能没有逆元，这段消元不再成立。需要根据题目使用整数消元、模素数分解或其他同余处理，不能换个模数直接复用。

11.5 0 1 分数规划

最大化 Σv/Σw 时，试阈值 x，问题转成是否存在选择使 Σ(v−xw)≥0。分母必须始终为正；选固定 k 项时只需取变换后最大的 k 个数求和。

复杂度与接口 O(迭代次数×n log n)，用 nth_element 可降平均排序开销。

```
double ratio_max(const vector<double> &a, const vector<double> &b, int keep) {
    assert(keep >= 1 && keep <= (int)a.size());
    double lo = 0, hi = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        assert(b[i] > 0);
        hi = max(hi, a[i] / b[i]);
    }
    for (int rep = 0; rep < 80; rep++) {
        double mid = (lo + hi) / 2;
        vector<double> v(a.size());
        for (int i = 0; i < (int)a.size(); i++)
            v[i] = a[i] - mid * b[i];
        sort(v.begin(), v.end(), greater<double>());
        double s = accumulate(v.begin(), v.begin() + keep, 0.0);
        if (s >= 0)
            lo = mid;
        else
            hi = mid;
    }
    return lo;
}
```

实现要点 分母须为正或所有可选方案的分母和严格为正。这个模型的 check 使用贪心排序，不是普通背包。

check 的单调性来自分母正和阈值变大使每项变差。若选择数可变或还有连通性约束，判定本身可能需要 DP、最优树或其他算法。

本实现假设 a≥0，因此二分下界为 0。若 a 可为负，应把下界调整为各单项比值的最小值。百分数输出与四舍五入按题目误差要求处理。

11.6 概率 DP 与期望方程

概率题先区分“状态出现的概率”和“从状态开始的期望代价”。前者常向后分配概率质量，后者常从后继状态反推。只有独立事件才可以直接相乘，但期望的线性性不要求独立。

若从状态 i 一次付出代价 c_i，再以 p_ij 进入 j，则 E_i=c_i+Σp_ijE_j。状态无环时按逆拓扑序求；有自环时把 p_iiE_i 移到左侧；有多状态循环时可能要解线性方程。若有策略选择，对每个可行动作分别计算期望，再取最优。

正确性 全期望公式按下一步的互斥完备结果分解。若目标状态以概率一到达且期望有限，方程对应期望值；不能只看到方程可写就假设唯一有限解。自环概率为 1 且每步正代价时，期望是无穷。

复杂度与接口 无环图 O(n+m)；一般稠密方程高斯消元 $O(n^3)$。以下为每步以 p 成功终止、否则留在原状态的特例。

```
long double geometric_expectation(long double p, long double one_step_cost) {
    assert(p > 0 && p <= 1);
    return one_step_cost / p;
}
```

实现要点 “期望花费不超过预算”与“每个随机分支花费都不超过预算”是不同约束。后者必须先检查所有可能后继的可行性，不能用平均值代替。概率接近 1 的自环会放大数值误差。

11.7 SG 与经典取石子游戏

无合法动作的一方输、状态无环的公平组合游戏中，SG 是所有后继 SG 集合的 mex。独立子游戏合并时 SG 异或为零是先手必败，否则存在走法使异或归零。规则是否允许拆堆与状态是否无环是使用前提。

复杂度与接口 有限取数集合 M、总石子 N 的 DP 为 O(N|M|)。

```
vector<int> sg_table(int N, const vector<int> &moves) {
    vector<int> sg(N + 1);
    for (int x = 1; x <= N; x++) {
        vector<bool> seen(moves.size() + 1);
        for (int d : moves)
            if (d <= x && sg[x - d] < (int)seen.size())
                seen[sg[x - d]] = true;
        while (sg[x] < (int)seen.size() && seen[sg[x]])
            ++sg[x];
    }
    return sg;
}
bool misere_nim(const vector<ll> &a) {
    ll x = 0;
    int ones = 0;
    bool large = false;
    for (ll v : a) {
        x ^= v;
        ones += v == 1;
        large |= v > 1;
    }
    return large ? x != 0 : ones % 2 == 0;
}
```

实现要点 有环局面可能存在和局，不能直接递归 SG。逆 Nim、斐波那契博弈与威佐夫的规则不同，必须匹配终止与取数条件。

取石子题应先比较合法操作，而非只根据题材选公式。有环状态可能出现和局，需按胜负逆推处理；无环公平组合游戏才适用递归 SG。

威佐夫败态为 $(\lfloor k\varphi \rfloor, \lfloor k\varphi \rfloor + k)$，大整数要用精确比较避免黄金分割浮点误差。斐波那契取石子的首次不能取完、随后最多取上次两倍，是 Zeckendorf 最小项结论的前提。E 与 D 拆堆和各种 SG 拓展仍应先列出合法后继，再归纳规律并证明。

11.8 有环图的胜负逆推

终点没有合法走法，是败态；一个节点只要有一步到败态，就是胜态；全部合法走法都到胜态，才是败态。在反图中维护每点尚未确定为胜态的后继数量，可从终点开始传播。

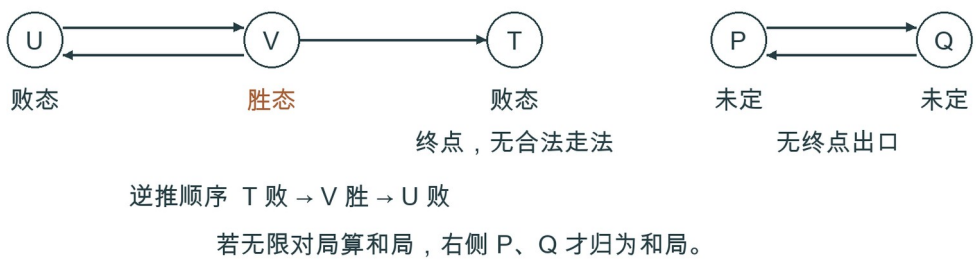


图 11.2 含环状态中的胜负与和局

复杂度与接口 O(n+m)，空间 O(n+m)。

```
vector<int> game_states(const vector<vector<int>> &g) {
    int n = g.size();
    vector<vector<int>> rev(n);
    vector<int> left(n), s(n, -1);
    queue<int> q;
    for (int u = 0; u < n; u++) {
        left[u] = g[u].size();
        for (int v : g[u])
            rev[v].push_back(u);
        if (!left[u])
            s[u] = 0, q.push(u);
    }
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int u : rev[v])
            if (s[u] < 0) {
                if (s[v] == 0)
                    s[u] = 1, q.push(u);
                else if (--left[u] == 0)
                    s[u] = 0, q.push(u);
            }
    }
    return s; // 0 败, 1 胜, -1 和局。
}
```

实现要点 有环状态不能直接记忆化 SG。终止条件和当前行动方必须建模完整；本节代码默认无合法行动者败。

剩余未确定节点可能形成可无限循环的局面，通常按题目规则为和局。存在强制步数上限、不同终止规则或非零和目标时，分类需要调整。

11.9 从规则选择博弈与线性模型

Nim 比较堆大小的异或，逆 Nim 需单独处理所有非空堆均为一的边界；巴什游戏按取数上限取模。使用任何结论前，都应核对末粒胜负、每步取数范围与拆堆规则。

同余消元、异或消元和实数消元的运算数域不同；开关问题常是 GF2，周期排班常是素数模，几何参数常是实数。向量相关性与方程可解性有相同的消元结构，但输出含义不同。

11.10 典型题解析

例题 11.1 Xiangqi 有环博弈与和局

2025 ICPC 南京 K

问题 棋盘上有车和马，马先手，判断车是否能在有限步数内抓住马。

解法 最稳妥的通用模型是 (车位置,马位置,行动方)。终止状态按赢家标记，其他状态连接一步合法走法。

沿反图传播：当前行动方能走到对手的败态，就能获胜；所有后继都让对手获胜，则当前必败。

仍无法判定的状态处在可持续循环中，是和局而不是胜或败。

原题的特殊棋盘规则可进一步推出“一步能抓住则胜，否则无法强制抓住”的结论。练习通用逆推时，仍应保留行动方并完整构造状态转移。

正确性 胜态只需一个取胜后继，败态必须排除所有逃生后继，所以分别用“存在”和“全部”两种传播规则。未决有环区域允许持续对弈，SG 无法直接处理。

示例推演 三个状态 U→V、V→U、V→终止败态。V 因可走到败态而成为胜态，U 所有后继均为胜，故败；如果删除终止边，U、V 都保持和局。

实现要点 把递归访问到环直接当败态会错。当前行动方也在状态里，即便车马位置相同，轮到谁走也会改变含义。

复杂度 通用逆推 O(状态数+转移数)，原题简化判定更小。

练习建议 先在手画的小有向图上验证胜败和三种结果，再模拟小棋盘枚举合法走法；最终读原题规则核对简化结论。

原题与解法依据 2025 ICPC 南京 K

例题 11.2 Chi Fan 期望 DP 与预算可行性

2025 ICPC 南京 I

问题 每天可独自或和队友吃饭，支出和满意度不同。队友此前是否支付过车费影响本次由谁付款；要求所有可能分支都不超预算，最大化满意度期望。

解法 状态 f[day][budget][A][B] 表示从这天开始、剩余预算固定、两个队友此前付费标记固定时的最大未来期望。

倒序枚举日期。每种决策先检查所有随机后继的支付是否能被预算覆盖，再计算收益加后继期望。

只要某个正概率分支无法保证后续预算，就不能把该决策视为可行；可行决策之间才取 max。

滚动日期维度，只保留当天和下一天，避免 n×m×4 的全部表。

正确性 标记 A、B 总结了未来费用分配所需的全部历史；给定状态后随机后继只由当日选择与规则决定。倒序 DP 让“未来是否够钱”成为已知条件，而非用平均花费替代保证。

示例推演 某决策一半概率花 0、一半概率花 10，当前预算 6。平均花费 5 看似够，但实际有正概率超支，必须判为不可行。这个小例子解释为何先判断全部分支再算期望。

实现要点 不能用“期望支出≤预算”代替每条路径可行。概率为 0 的分支不构成预算限制；不可行状态不要初始化成 0 后参与 max。

复杂度 O(nm) 时间，滚动后 O(m) 空间，状态常数为 4。

练习建议 先把随机分支限定为两个，写完整策略树暴力，再核对倒序 DP；最后处理精度误差与原题概率。

原题与解法依据 2025 ICPC 南京 I

第 12 章　基础几何与凸包

几何算法首先需要可靠的方向与位置判定：点积处理投影和夹角，叉积处理转向和面积，随后才是凸包、极角排序与扫描。重复点、共线与端点接触应在模型中统一定义。

12.1　整数向量与凸包

Andrew 凸包按坐标排序，分别维护下链和上链。若新点使末尾两点不再保持规定转向，就删除中间点。每点至多入栈出栈一次，排序占主导复杂度。

向量 u、v 的点积为 ux·vx+uy·vy，叉积为 ux·vy−uy·vx。非零向量点积为零表示垂直；叉积为零表示平行，同射线还需点积为正。代码 dot(a,b,c) 与 cross(a,b,c) 分别作用于以 a 为共同起点的两个向量，避免误把位置坐标直接当方向。

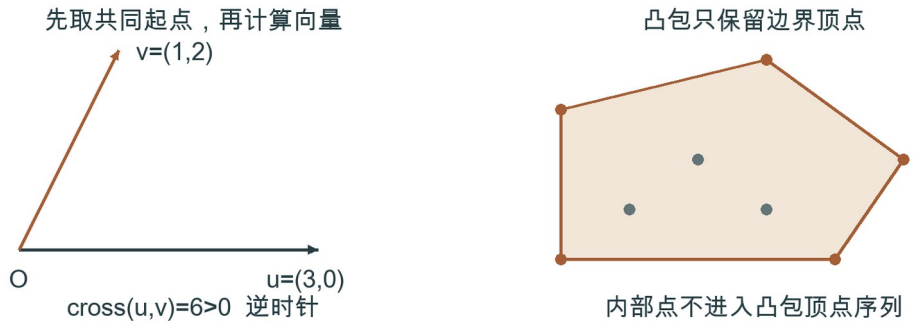


图 12.1　向量转向与凸包顶点

复杂度与接口　凸包 O(n log n)，空间 O(n)。

```
struct Point {
    ll x, y;
    bool operator<(const Point &b) const { return tie(x, y) < tie(b.x, b.y); }
    bool operator==(const Point &b) const { return x == b.x && y == b.y; }
};
i128 cross(Point a, Point b, Point c) {
    return ((i128)b.x - a.x) * ((i128)c.y - a.y) -
           ((i128)b.y - a.y) * ((i128)c.x - a.x);
}
vector<Point> hull(vector<Point> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = p.size();
    if (n <= 1)
        return p;
    vector<Point> h;
    for (auto x : p) {
        while (h.size() > 1 && cross(h[h.size() - 2], h.back(), x) <= 0)
            h.pop_back();
        h.push_back(x);
    }
    int base = h.size();
```

```
    for (int i = n - 2; i >= 0; i--) {
        while ((int)h.size() > base &&
               cross(h[h.size() - 2], h.back(), p[i]) <= 0)
            h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h;
}
i128 dot(Point a, Point b, Point c) {
    return (i128(b.x) - a.x) * (i128(c.x) - a.x) +
           (i128(b.y) - a.y) * (i128(c.y) - a.y);
}
```

实现要点 叉积先提升到 i128 再做坐标差的乘法。该版删除边上的中间共线点，全共线返回两个端点；不同题目可能要保留共线边界点。

保留边界共线点与仅保留顶点，对应不同的弹栈条件。本章凸包返回不含边中间点的顶点序列，便于旋转卡壳；空输入、单点与全共线输入须分别约定返回结果。

12.2 线段相交与旋转卡壳

整数几何优先使用叉积与点积，把方向、共线和投影关系变成符号判断。线段相交包括端点与重叠两类退化情况；凸包直径可以利用对踵点单调移动，在凸包上用线性扫描替代全点对枚举。

两线段端点分别位于对方所在直线的两侧时相交；叉积为零时另检查点是否处于线段包围盒。旋转卡壳沿凸包边前进，对面点到当前边的有向面积先增后减，保留最大面积位置，比较相应端点距离。

正确性 凸多边形的支撑线方向连续转动，对踵点不会倒退。每个指针最多绕一圈，故扫描线性。相交判定先处理共线边界，再用严格异侧，避免将延长线相交误判为线段相交。

复杂度与接口 线段相交 O(1)；凸包后求直径 O(h)，总复杂度 O(n log n)。依赖本章 Point、cross 与凸包返回逆时针且不含边中间共线点。

```
int sign128(i128 x) {
    return (x > 0) - (x < 0);
}
bool on_segment(Point a, Point b, Point p) {
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x &&
           p.x <= max(a.x, b.x) && min(a.y, b.y) <= p.y &&
           p.y <= max(a.y, b.y);
}
bool intersects(Point a, Point b, Point c, Point d) {
    i128 x = cross(a, b, c), y = cross(a, b, d);
    i128 z = cross(c, d, a), w = cross(c, d, b);
    if (!x && on_segment(a, b, c))
        return true;
    if (!y && on_segment(a, b, d))
        return true;
    if (!z && on_segment(c, d, a))
        return true;
    if (!w && on_segment(c, d, b))
        return true;
```

```
    return sign128(x) * sign128(y) < 0 && sign128(z) * sign128(w) < 0;
}
i128 distance2(Point a, Point b) {
    i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y;
    return x * x + y * y;
}
i128 hull_diameter(const vector<Point> &p) {
    int n = p.size();
    if (n <= 1)
        return 0;
    if (n == 2)
        return distance2(p[0], p[1]);
    i128 ans = 0;
    int j = 1;
    for (int i = 0; i < n; ++i) {
        int k = (i + 1) % n;
        while (cross(p[i], p[k], p[(j + 1) % n]) > cross(p[i], p[k], p[j]))
            j = (j + 1) % n;
        ans = max(ans, max(distance2(p[i], p[j]), distance2(p[k], p[j])));
    }
    return ans;
}
```

实现要点 坐标差要先提升再相减，避免在 ll 中已溢出；坐标允许接近整个 ll 范围时 i128 的平方仍可能溢出。浮点几何的 eps 应与数据尺度匹配，不能把固定 eps 当作通用正确性保证。

12.3 极角面积与退化情况

整数极角排序先按上半平面与下半平面分组，再用叉积比较角度，同方向按长度或任务语义处理。直接使用 atan2 方便但引入浮点误差；交叉乘积本身也可能溢出，必须统一数值契约。零向量没有方向，应先移除或单独定义。

鞋带公式给有向面积，取绝对值后才是面积。相邻重复点、全部共线、端点接触和线段重叠要分别确认。距离比较优先比较平方值，只有最终输出长度时再开方；圆与直线问题涉及实数时再引入适合尺度的误差策略。

12.4 典型题解析

例题 12.1 Grass 从五点结论到可靠几何实现

2022 CCPC 威海 C

问题 从平面点中选 5 个，其中一个中心向其余 4 点连线，要求两条线段除中心外不相交。

解法 五点全部共线则不可能；命题组证明任意不全共线五点总有一个可用中心。先找到这样一组，之后只需常数规模检查。

对某个候选中心 O，检查其余点两两：cross(O,A,B)=0 且 dot(A−O,B−O)>0 才表示同一射线，会有线段重叠。

逐一尝试五个候选中心即可。题目保证输入点互异；若推广到允许重复点的情形，需先去重，再核对是否仍有至少五个点。

固定四点，若它们已经不全共线，任加第五点即可；否则扫描一个不在该直线上的点。

正确性 关键五点存在性结论可分为无三点共线、四点共线、恰有三点共线三类；后两类都能选线外点或中间点避免同射线。实现只需验证具体中心，证明与代码可以分开。

示例推演 四点在 x 轴上，再加一点 (0,1)。以 (0,1) 为中心，向 x 轴上四个互异点的线段只在中心相交。若选 x 轴端点为中心，其他轴上点可能在同射线。

实现要点 共线不等于重叠：中心两侧的反向射线合法。坐标差乘法使用 i128；不要用浮点角度比较同射线。

复杂度 O(n) 找五点，O(1) 验证，额外空间 O(1)。

练习建议 用整数坐标生成无共线、三点共线、四点共线、全共线数据。画出反向射线作为必测反例。

原题与解法依据 2022 CCPC 威海 C

例题 12.2 What More Kangaroos 极角覆盖与远点缩放

2025 ICPC 南京 B

问题 若干袋鼠的同步操作可转为不等式 a[i]x+b[i]y+c[i]<0，其中 c[i]>0，找整点使满足的不等式尽量多。

解法 固定一个非零方向 (x,y)，沿该方向放大 t。若 a[i]x+b[i]y<0，则充分远处可以抵消正 c[i]；反之永远不满足。

因此 c[i] 只影响需要走多远，不影响方向是否最终可行。问题变为最大数量法向量与目标方向点积严格为负。

每个不等式对应圆周上的一个开半圆，极角排序并将序列复制一次，用双指针找严格半圆内最多法向量。

同方向与反方向要精确分组。用象限、叉积比较角度，端点严格性用点积及叉积判断，不使用粗糙 EPS。

正确性 方向放大不改变齐次部分符号。所有可行方向组成有限个开区间，而有理斜率在开区间中稠密，可取整数方向，充分放大后同时满足其对应的不等式。

示例推演 x+2y+3<0 与 −x+y+5<0。方向 (−1,−1) 的齐次项分别为 −3 与 0，放大只能满足第一条；方向 (−1,−2) 分别为 −5 与 −1，足够大时两条都满足。

实现要点 齐次项等于 0 时 c>0，严格不等式必定不成立。a=b=0 的式子直接忽略。角度窗口跨过 2π 必须复制或循环处理。

复杂度 O(n log n) 时间，O(n) 空间。

练习建议 先练排序整数方向，再以少量向量枚举所有临界方向两侧取样核对窗口答案。

原题与解法依据 2025 ICPC 南京 B